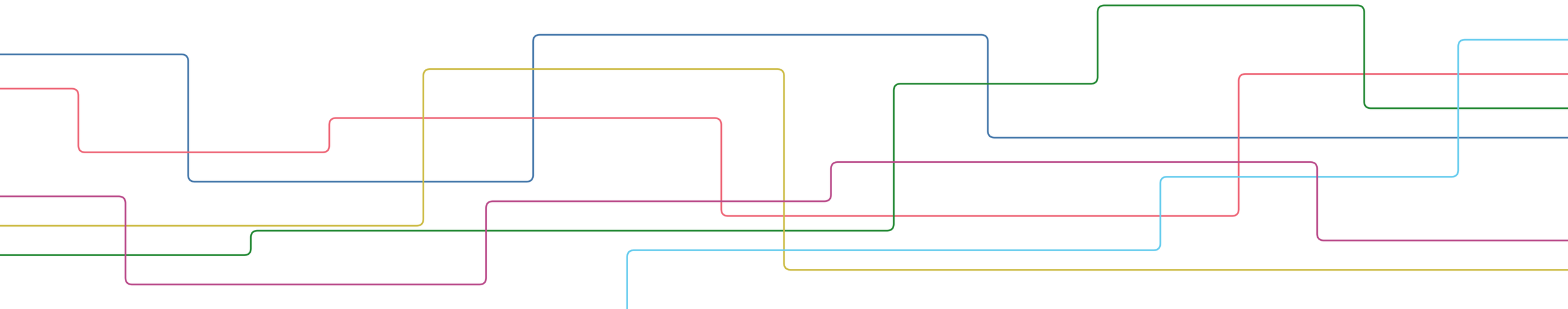
Programming for Data Science

– Linear Models and the Kernel Trick

Martin Trapp

Department of Computing and Learning Systems
School of Electrical Engineering and Computer Science
KTH Royal Institute of Technology
mtrapp@kth.se



Outline

Linear Regression

- Squared loss
- Optimal weights in linear regression
- Predictions in linear regression

Kernel Trick

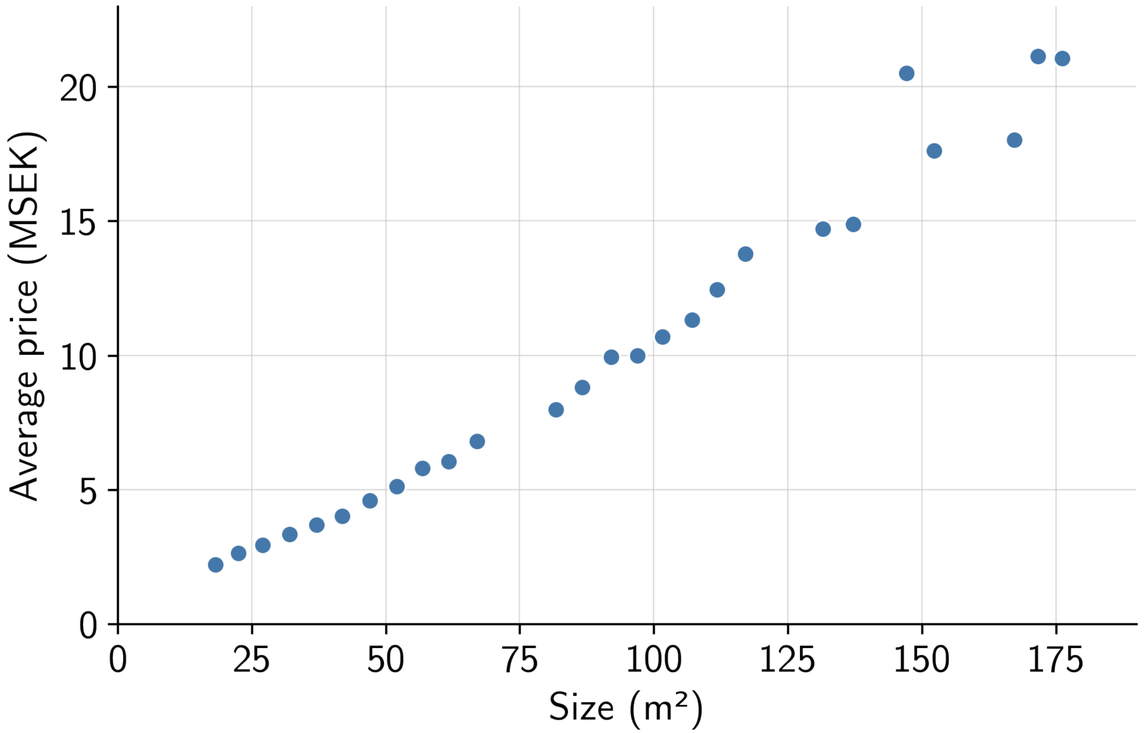
- Inner products and the kernel trick

Kernel Ridge Regression

- Kernel regression
- Regularising kernel regression

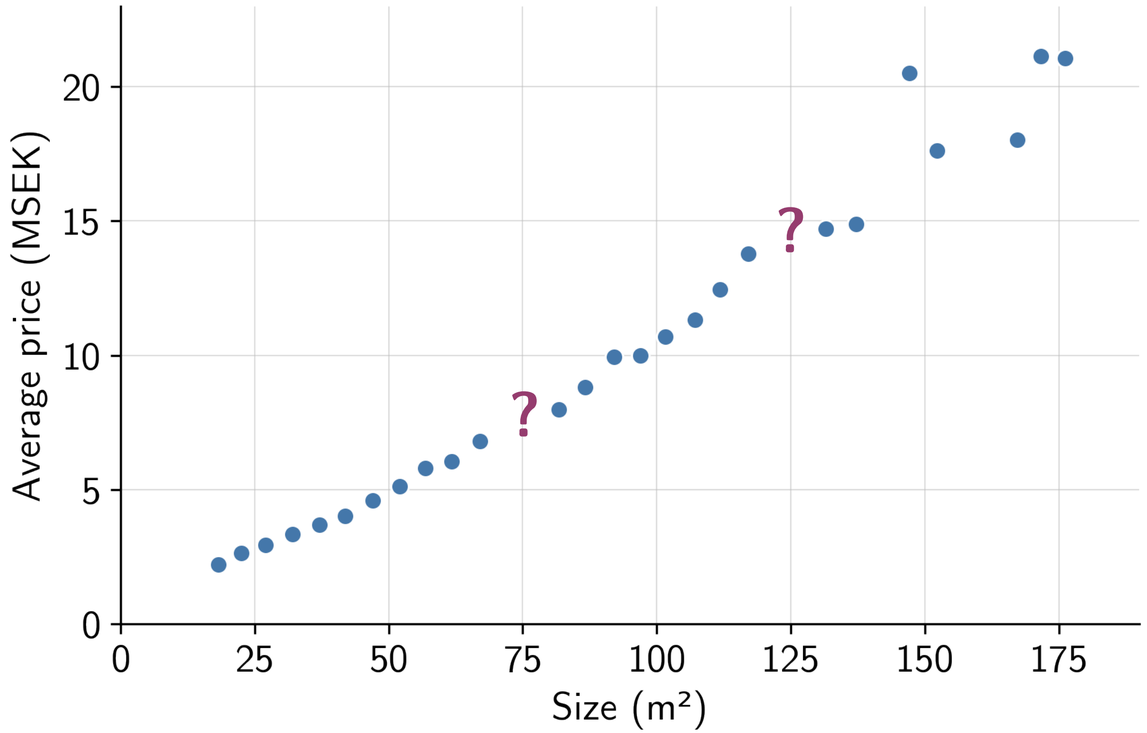
Regression problems

- Suppose we are given a data set of house prices:



Regression problems

- Suppose we are given a data set of house prices:



- **Goal:** Build a model to predict the avg. prices of a house of 70m² (?) and 160m² (?).

Regression problems

- Suppose we are given a data set: $\mathscr{D} = \{(x_n, y_n)\}_{n=1}^{N}$

- House example: x_n = house area and y_n = house price

Regression problems

- Suppose we are given a data set: $\mathscr{D} = \{(x_n, y_n)\}_{n=1}^{N}$
- House example: $x_n =$ house area and $y_n =$ house price
- **Goal:** Learn a function f such that: $y_n = f(x_n) + \epsilon_n$ where ϵ_n is observation noise

Regression problems

- Suppose we are given a data set: $\mathscr{D} = \{(x_n, y_n)\}_{n=1}^{N}$
- House example: $x_n =$ house area and $y_n =$ house price
- **Goal:** Learn a function f such that: $y_n = f(x_n) + \epsilon_n$ where ϵ_n is observation noise
- What should f look like?

Regression problems

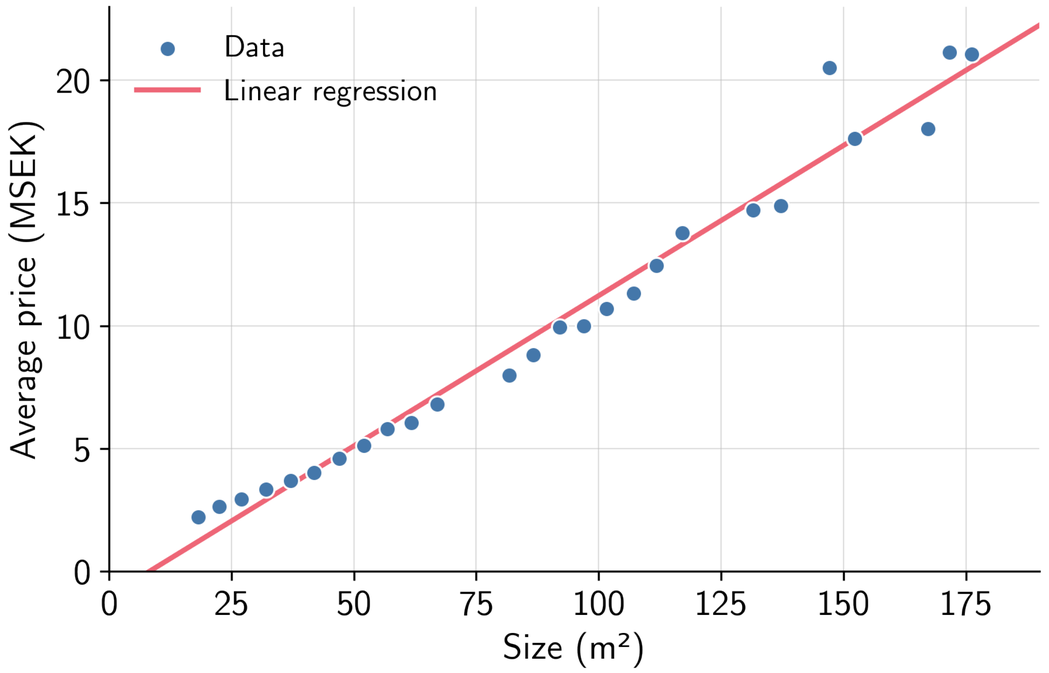
- Suppose we are given a data set: $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^{N}$
- House example: x_n = house area and y_n = house price
- **Goal:** Learn a function f such that: $y_n = f(x_n) + \epsilon_n$ where ϵ_n is observation noise
- What should f look like?
- **Linear Regression:** f could be a *linear* function:

$$f(\mathbf{x}) = x_1 w_1 + x_2 w_2 + \cdots = \sum_{i=1}^{D} x_i w_i$$

Linear Regression

- **Linear Regression:** f is *linear* in the parameters and inputs!

$$f(\mathbf{x}) = x_1 w_1 + x_2 w_2 + \cdots = \sum_{i=1}^{D} x_i w_i$$



Linear Regression

- **Linear Regression:** f is *linear* in the parameters and inputs!

$$f(\mathbf{x}) = x_1 w_1 + x_2 w_2 + \cdots = \sum_{i=1}^{D} x_i w_i$$

- What values does $\mathbf{w} = [w_1, \ldots, w_D]^T$ have?

?

Linear Regression

- **Recall:** Our goal is to find $y_n = f(x_n) + \epsilon_n$ with

$$f(\mathbf{x}) = x_1 w_1 + x_2 w_2 + \cdots = \sum_{i=1}^{D} x_i w_i$$

- **Squared Loss:**

$$\mathscr{L}(\mathbf{w}) = \sum_{n=1}^{N} \left(y_n - \mathbf{w}^T \mathbf{x}_n \right)^2$$

Linear Regression

- **Recall:** Our goal is to find $y_n = f(x_n) + \epsilon_n$ with

$$f(\mathbf{x}) = x_1 w_1 + x_2 w_2 + \cdots = \sum_{i=1}^{D} x_i w_i$$

- **Squared Loss:**

$$\mathscr{L}(\mathbf{w}) = \sum_{n=1}^{N} \left(y_n - \mathbf{w}^T \mathbf{x}_n\right)^2$$

$$\mathscr{L}(\mathbf{w}) = \left(\mathbf{y} - \mathbf{X}\mathbf{w}\right)^T \left(\mathbf{y} - \mathbf{X}\mathbf{w}\right)$$

Matrix notation

$[\mathbf{X}\mathbf{w}]_n = \mathbf{x}_n^T \mathbf{w}$

Linear Regression

- **Recall:** Our goal is to find $y_n = f(x_n) + \epsilon_n$ with

$$f(\mathbf{x}) = x_1 w_1 + x_2 w_2 + \cdots = \sum_{i=1}^{D} x_i w_i$$

- **Squared Loss:**

$$\mathscr{L}(\mathbf{w}) = \sum_{n=1}^{N} \left(y_n - \mathbf{w}^T \mathbf{x}_n\right)^2$$

Matrix notation

$[\mathbf{X}\mathbf{w}]_n = \mathbf{x}_n^T \mathbf{w}$

$$\mathscr{L}(\mathbf{w}) = \left(\mathbf{y} - \mathbf{X}\mathbf{w}\right)^T \left(\mathbf{y} - \mathbf{X}\mathbf{w}\right)$$

Multiply out

$$\mathscr{L}(\mathbf{w}) = \mathbf{y}^T\mathbf{y} - 2\,\mathbf{w}^T\mathbf{X}^T\mathbf{y} + \mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w}$$

Linear Regression

- **Recall:** Our goal is to find $y_n = f(x_n) + \epsilon_n$ with

$$f(\mathbf{x}) = x_1 w_1 + x_2 w_2 + \cdots = \sum_{i=1}^{D} x_i w_i$$

- **Squared Loss:**

$$\mathscr{L}(\mathbf{w}) = \sum_{n=1}^{N} \left(y_n - \mathbf{w}^T \mathbf{x}_n\right)^2$$

Matrix notation

$$[\mathbf{X}\mathbf{w}]_n = \mathbf{x}_n^T \mathbf{w}$$

$$\mathscr{L}(\mathbf{w}) = \left(\mathbf{y} - \mathbf{X}\mathbf{w}\right)^T \left(\mathbf{y} - \mathbf{X}\mathbf{w}\right)$$

Multiply out

$$\mathscr{L}(\mathbf{w}) = \mathbf{y}^T\mathbf{y} - 2\,\mathbf{w}^T\mathbf{X}^T\mathbf{y} + \mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w}$$

Linear Regression

- **Optimal $\mathbf{w}^* = [w_1, \ldots, w_D]^T$** under squared loss:

$$\mathscr{L}(\mathbf{w}) = \mathbf{y}^T\mathbf{y} - 2\,\mathbf{w}^T\mathbf{X}^T\mathbf{y} + \mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w}$$

Linear Regression

- **Optimal** $\mathbf{w}^* = [w_1, \ldots, w_D]^T$ under squared loss:

$$\mathscr{L}(\mathbf{w}) = \mathbf{y}^T\mathbf{y} - 2\,\mathbf{w}^T\mathbf{X}^T\mathbf{y} + \mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w}$$

Set gradient to zero

$$\nabla_{\mathbf{w}}\mathscr{L}(\mathbf{w}^*) = \mathbf{0} \quad \Longleftrightarrow \quad \mathbf{X}^T\mathbf{X}\,\mathbf{w}^* = \mathbf{X}^T\mathbf{y}$$

Linear Regression

- **Optimal** $\mathbf{w}^* = [w_1, \ldots, w_D]^T$ under squared loss:

$$\mathscr{L}(\mathbf{w}) = \mathbf{y}^T\mathbf{y} - 2\,\mathbf{w}^T\mathbf{X}^T\mathbf{y} + \mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w}$$

Set gradient to zero

$$\nabla_{\mathbf{w}}\mathscr{L}(\mathbf{w}^*) = \mathbf{0} \quad \Longleftrightarrow \quad \mathbf{X}^T\mathbf{X}\,\mathbf{w}^* = \mathbf{X}^T\mathbf{y}$$

Rearrange terms

$$\mathbf{w}^* = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}$$

Linear Regression

- **Optimal** $\mathbf{w}^* = [w_1, \ldots, w_D]^T$ under squared loss:

$$\mathscr{L}(\mathbf{w}) = \mathbf{y}^T\mathbf{y} - 2\,\mathbf{w}^T\mathbf{X}^T\mathbf{y} + \mathbf{w}^T\mathbf{X}^T\mathbf{X}\mathbf{w}$$

Set gradient to zero

$$\nabla_{\mathbf{w}}\mathscr{L}(\mathbf{w}^*) = \mathbf{0} \quad \Longleftrightarrow \quad \mathbf{X}^T\mathbf{X}\,\mathbf{w}^* = \mathbf{X}^T\mathbf{y}$$

Rearrange terms

Optimal solution:

$$\mathbf{w}^* = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}$$

Implementing Linear Regression

Linear Regression

```python
import numpy as np

# prepare some synthetic data
X = np.array([ [1.0, 25.0], [2.5, 35.2], [5.0, 50.0], [4.5, 45.0] ])
y = np.array([0.2, 0.35, 1.0, 0.6])

# rescale to zero mean
y = y - y.mean()
X = X - X.mean(axis=0)

# compute optimal w
Xt = X.transpose()  # precompute transposed X
A = Xt @ X          # multiply Xt and X
b = Xt @ y          # multiply Xt and y

w_opt = np.linalg.solve(A, b)  # solve linear system A*w = b for w

# perform predictions
Xstar = np.array([ [3.0, 40.0] ])
yhat = (Xstar - X_mean) @ w_opt + y_mean  # same centring as training data
```

https://github.com/traml-lab/id2214-lectures/blob/main/lecture5-linear-regression.py

Linear Regression

- In linear regression, the **optimal weights are a summary statistic of the data**

$$w = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}$$

- When we make **predictions**, we compute a weighted sum of the data

Linear Regression

- In linear regression, the **optimal weights are a summary statistic of the data**

$$w = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}$$

- When we make **predictions**, we compute a weighted sum of the data

$$f(\mathbf{x}_\star) = \sum_{i=1}^{D} x_{i\star} w_i = \mathbf{x}_\star^T\mathbf{w}$$

Linear Regression

- In linear regression, the **optimal weights are a summary statistic of the data**

$$w = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}$$

- When we make **predictions**, we compute a weighted sum of the data

$$f(\mathbf{x}_\star) = \sum_{i=1}^{D} x_{i\star} w_i = \mathbf{x}_\star^T\mathbf{w}$$

$$f(\mathbf{x}_\star) = \mathbf{x}_\star^T\left[\left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}\right]$$

Recall: $w = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}$

Linear Regression

- In linear regression, the **optimal weights are a summary statistic of the data**

$$w = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}$$

- When we make **predictions**, we compute a weighted sum of the data

$$f(\mathbf{x}_\star) = \sum_{i=1}^{D} x_{i\star} w_i = \mathbf{x}_\star^T \mathbf{w}$$

Recall: $w = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}$

$$f(\mathbf{x}_\star) = \mathbf{x}_\star^T \left[\left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}\right]$$

Write out the sum

$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} \mathbf{x}_\star^T \left(\mathbf{X}^T\mathbf{X}\right)^{-1} \mathbf{x}_n \, y_n$$

Linear Regression

- In linear regression, the **optimal weights are a summary statistic of the data**

$$w = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}$$

- When we make **predictions**, we compute a weighted sum of the data

$$f(\mathbf{x}_\star) = \sum_{i=1}^{D} x_{i\star} w_i = \mathbf{x}_\star^T \mathbf{w}$$

Recall: $w = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}$

$$f(\mathbf{x}_\star) = \mathbf{x}_\star^T \left[\left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}\right]$$

Write out the sum

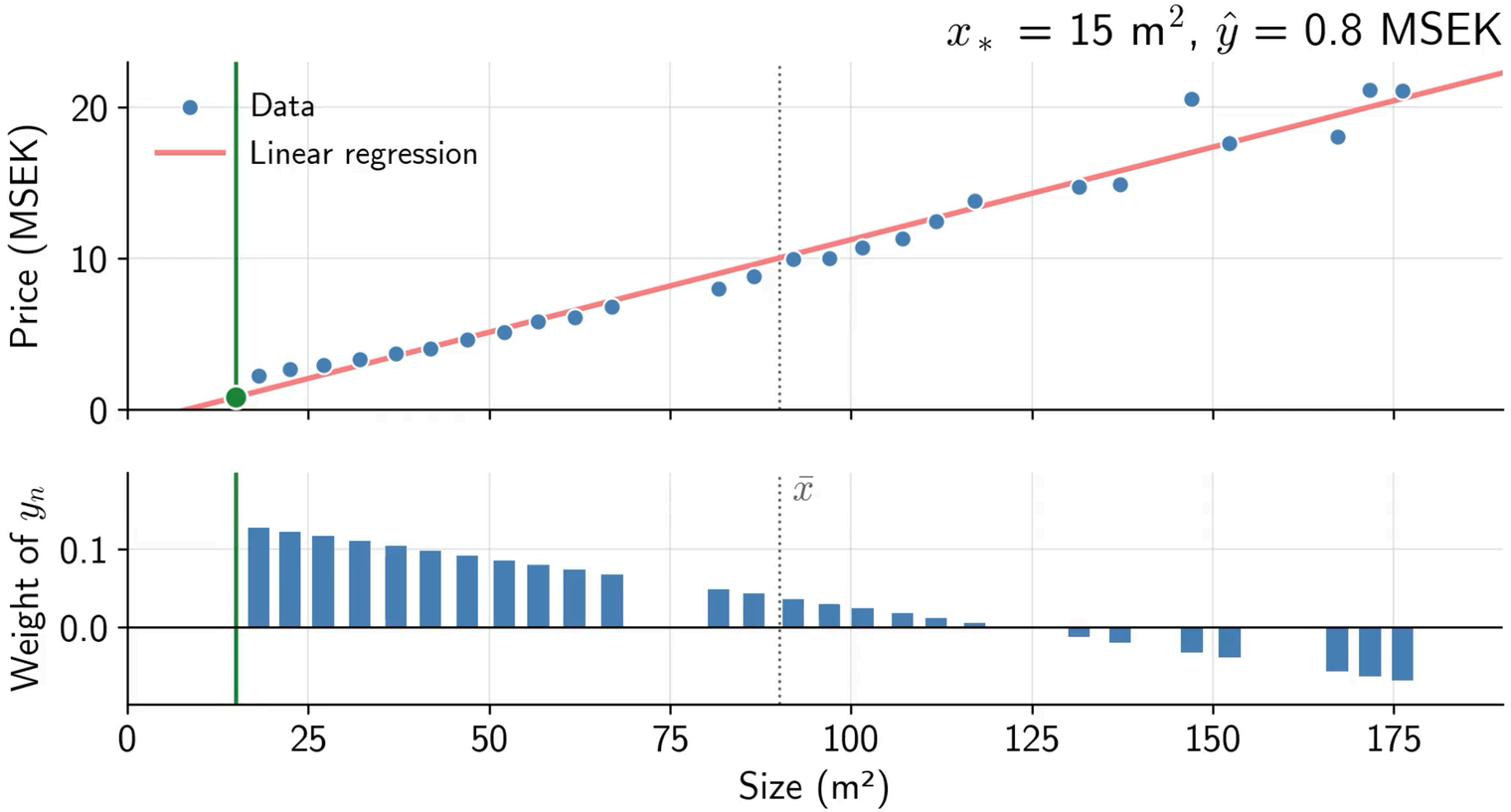
$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} \mathbf{x}_\star^T \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{x}_n\, y_n \qquad = \sum_{n=1}^{N} \alpha_n(\mathbf{x}_\star^T)\, y_n$$

Linear Regression

- When we make predictions, we compute a weighted sum of the data

$$\sum_{n=1}^{N} \alpha_n(\mathbf{x}_\star^T)\, y_n$$

$\alpha_n(\mathbf{x}_\star^T)$



Kernel trick

- Let's look more closely at how we compute predictions

$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} \alpha_n(\mathbf{x}_\star^T)\, y_n \qquad\qquad \alpha_n(\mathbf{x}_\star) = \mathbf{x}_\star^T \left(\mathbf{X}^T\mathbf{X}\right)^{-1} \mathbf{x}_n$$

Kernel trick

- Let's look more closely at how we compute predictions

$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} \alpha_n(\mathbf{x}_\star^T)\, y_n$$

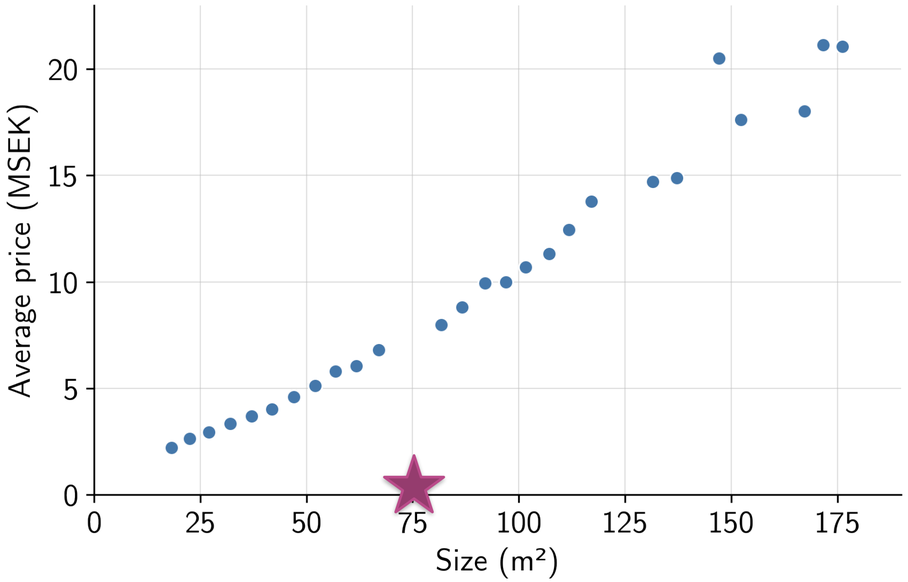
$$\alpha_n(\mathbf{x}_\star) = \mathbf{x}_\star^T \mathbf{S}^{-1} \mathbf{x}_n = \langle \mathbf{x}_\star, \mathbf{x}_n \rangle_{\mathbf{S}^{-1}}$$

Inner product between $\mathbf{x}_\star$ and $\mathbf{x}$

Kernel trick

- Let's look more closely at how we compute predictions

$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} \alpha_n(\mathbf{x}_\star^T)\, y_n \qquad \alpha_n(\mathbf{x}_\star) = \mathbf{x}_\star^T \mathbf{S}^{-1} \mathbf{x}_n$$

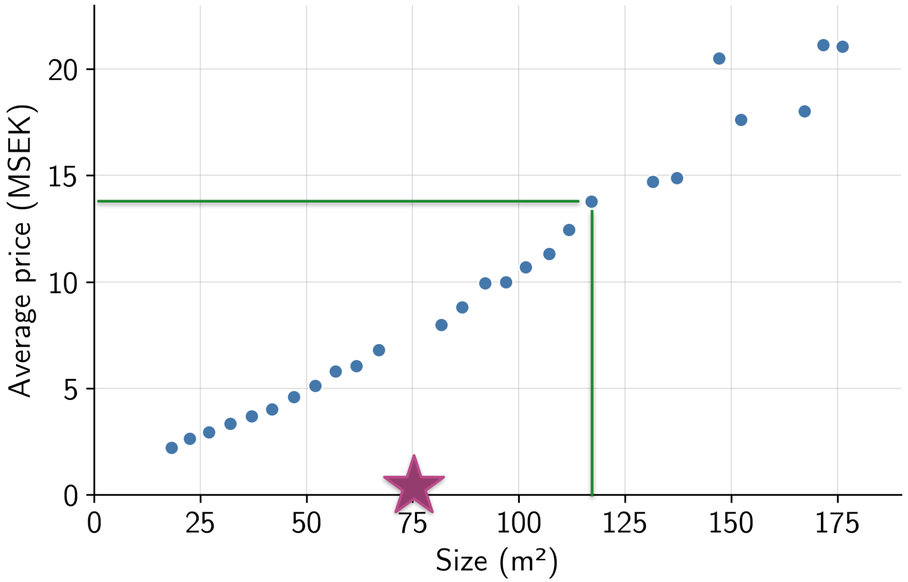


$x_\star = 75$

Kernel trick

- Let's look more closely at how we compute predictions

$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} \alpha_n(\mathbf{x}_\star^T)\, y_n \qquad \alpha_n(\mathbf{x}_\star) = \mathbf{x}_\star^T \mathbf{S}^{-1} \mathbf{x}_n$$



$x_\star = 75$

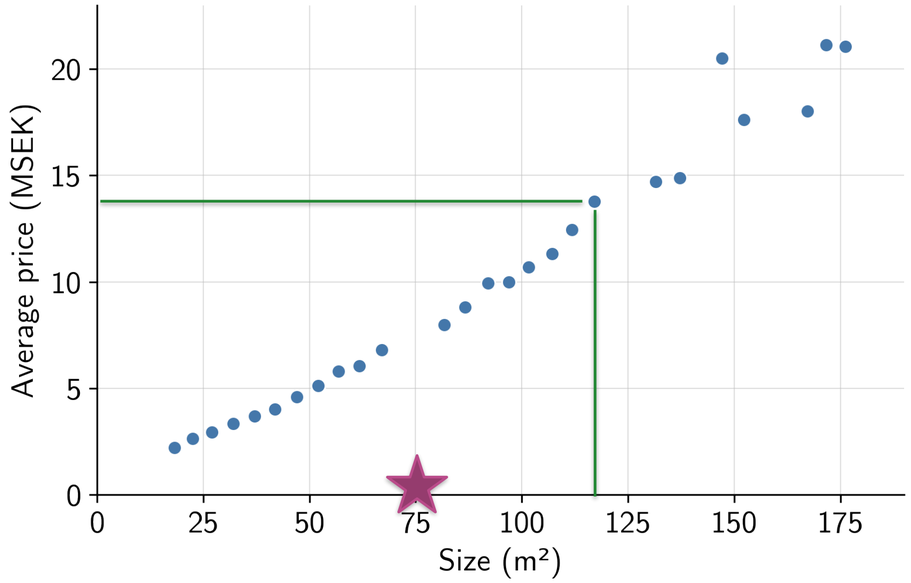
$x_n = 120$

$y_n = 14.5$

Kernel trick

- Let's look more closely at how we compute predictions

$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} \alpha_n(\mathbf{x}_\star^T)\, y_n \qquad \alpha_n(\mathbf{x}_\star) = \mathbf{x}_\star^T \mathbf{S}^{-1} \mathbf{x}_n$$



$x_\star = 75$

$x_n = 120$

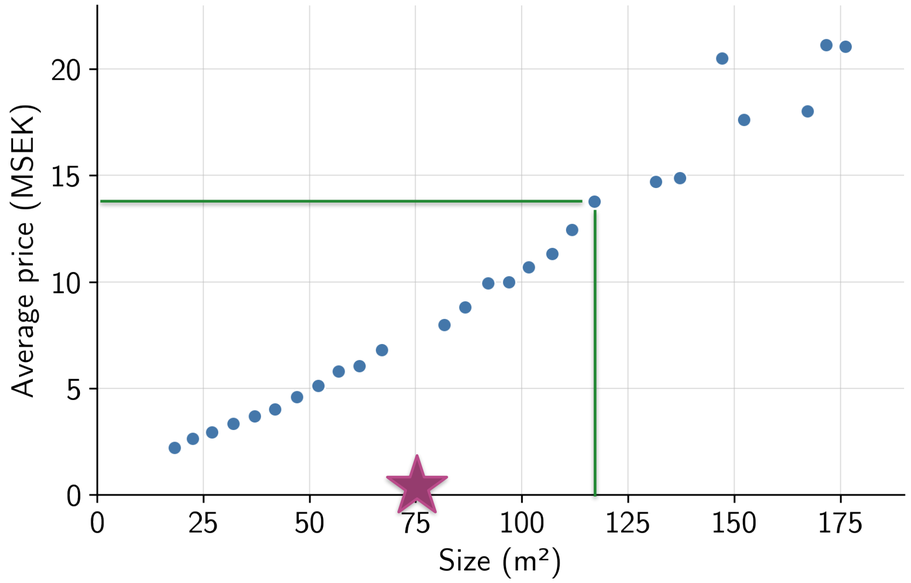
$y_n = 14.5$

$$S^{-1} = \left([x_1, \ldots, x_N] \begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix} \right)^{-1} = \left(\sum_{n=1}^{N} x_n^2 \right)^{-1}$$

Kernel trick

- Let's look more closely at how we compute predictions

$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} \alpha_n(\mathbf{x}_\star^T)\, y_n \qquad \alpha_n(\mathbf{x}_\star) = \mathbf{x}_\star^T \mathbf{S}^{-1} \mathbf{x}_n$$



$$x_\star = 75$$
$$x_n = 120$$
$$y_n = 14.5$$

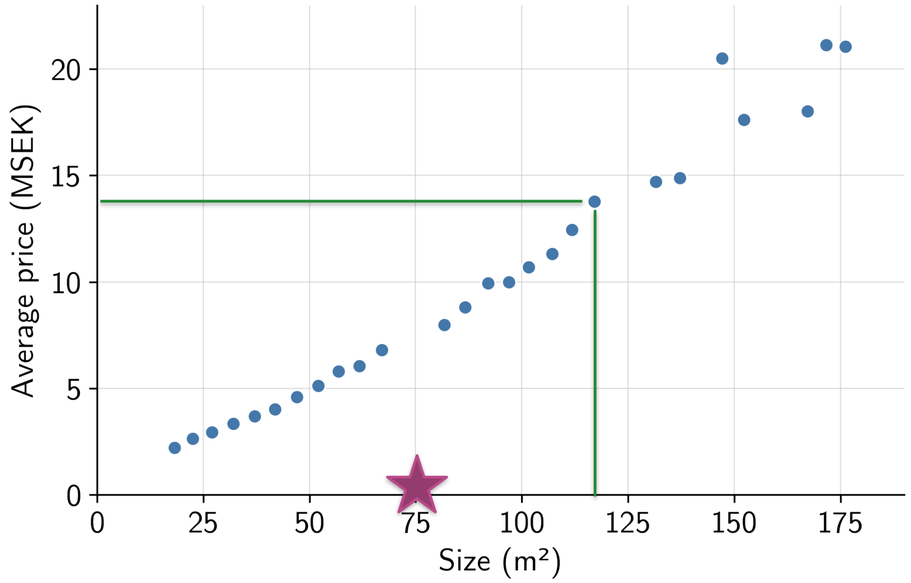
$$S^{-1} = \left([x_1, \ldots, x_N] \begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix} \right)^{-1} = \left(\sum_{n=1}^{N} x_n^2 \right)^{-1}$$

$$\alpha_n(x_\star) = \frac{75 \cdot 120}{\sum_n x_n^2}$$

Kernel trick

- Let's look more closely at how we compute predictions

$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} \alpha_n(\mathbf{x}_\star^T)\, y_n \qquad \alpha_n(\mathbf{x}_\star) = \mathbf{x}_\star^T \mathbf{S}^{-1} \mathbf{x}_n$$



$x_\star = 75$

$x_n = 120$

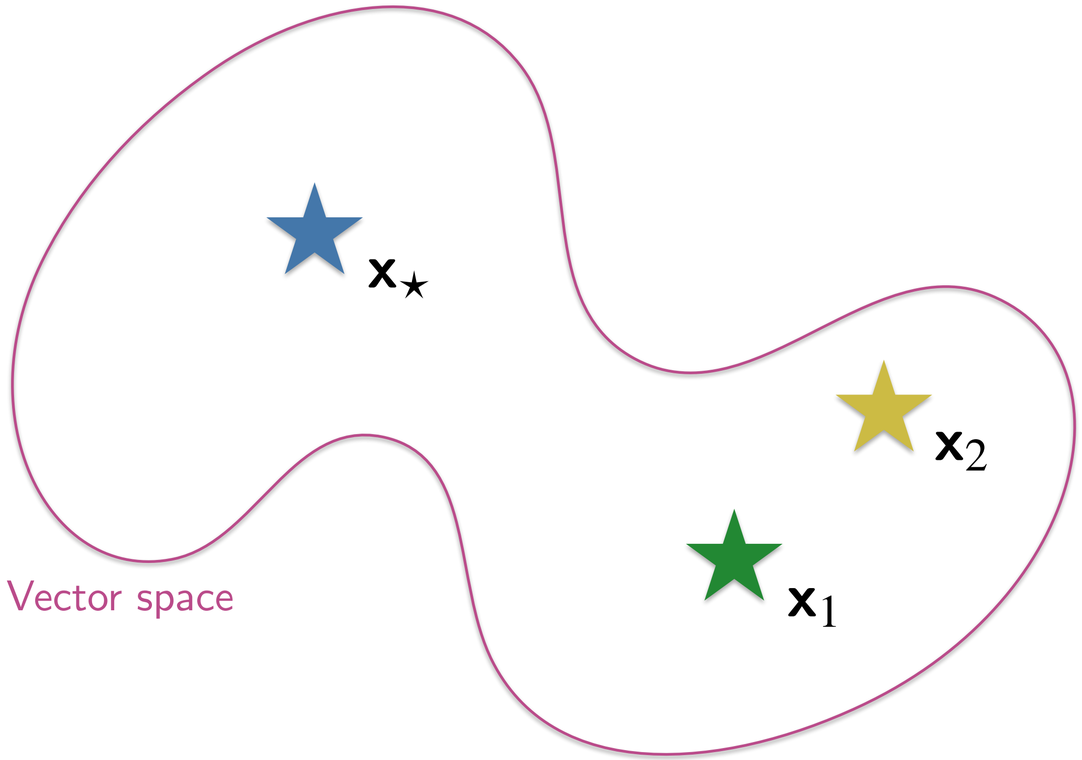
$y_n = 14.5$

$$S^{-1} = \left([x_1, \ldots, x_N] \begin{bmatrix} x_1 \\ \vdots \\ x_N \end{bmatrix} \right)^{-1} = \left(\sum_{n=1}^{N} x_n^2 \right)^{-1}$$

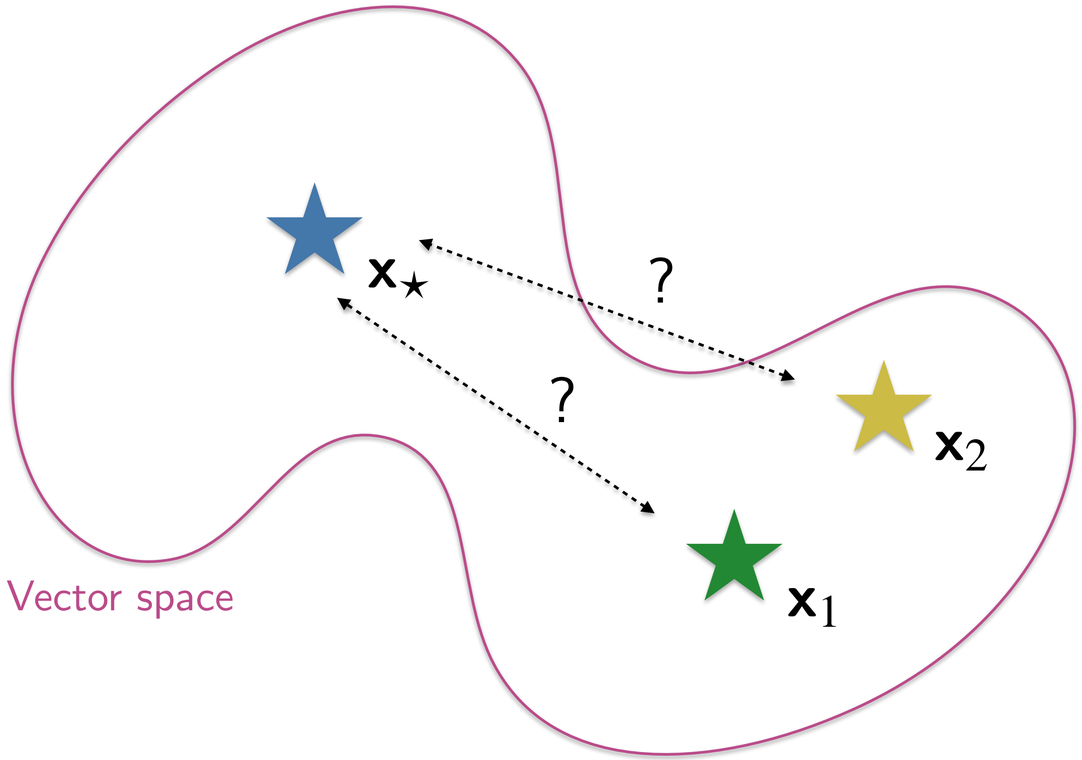
$$\alpha_n(x_\star) = \frac{75 \cdot 120}{\sum_n x_n^2}$$

The more similar $x_\star$ and x_n are, the larger α_n!

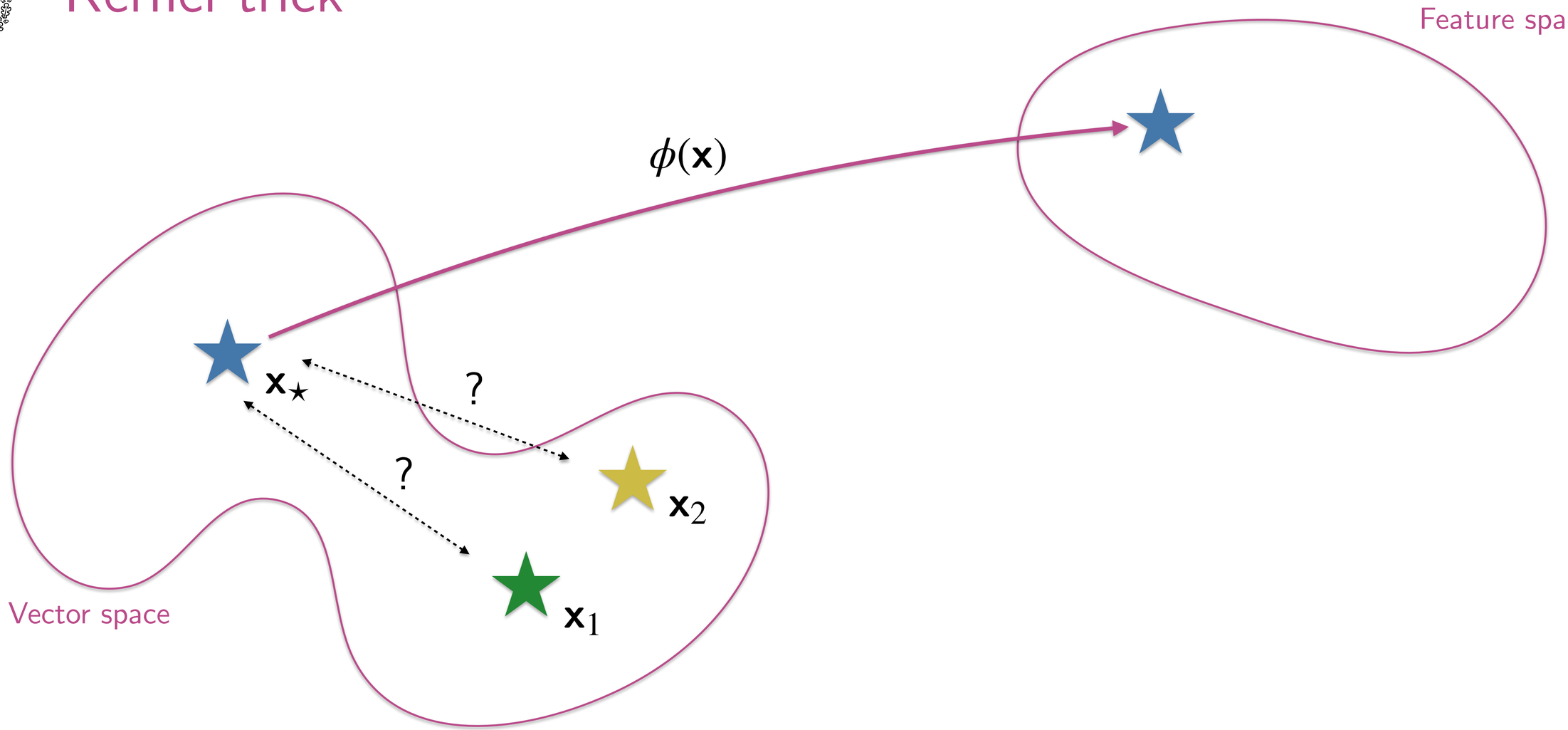
Kernel trick



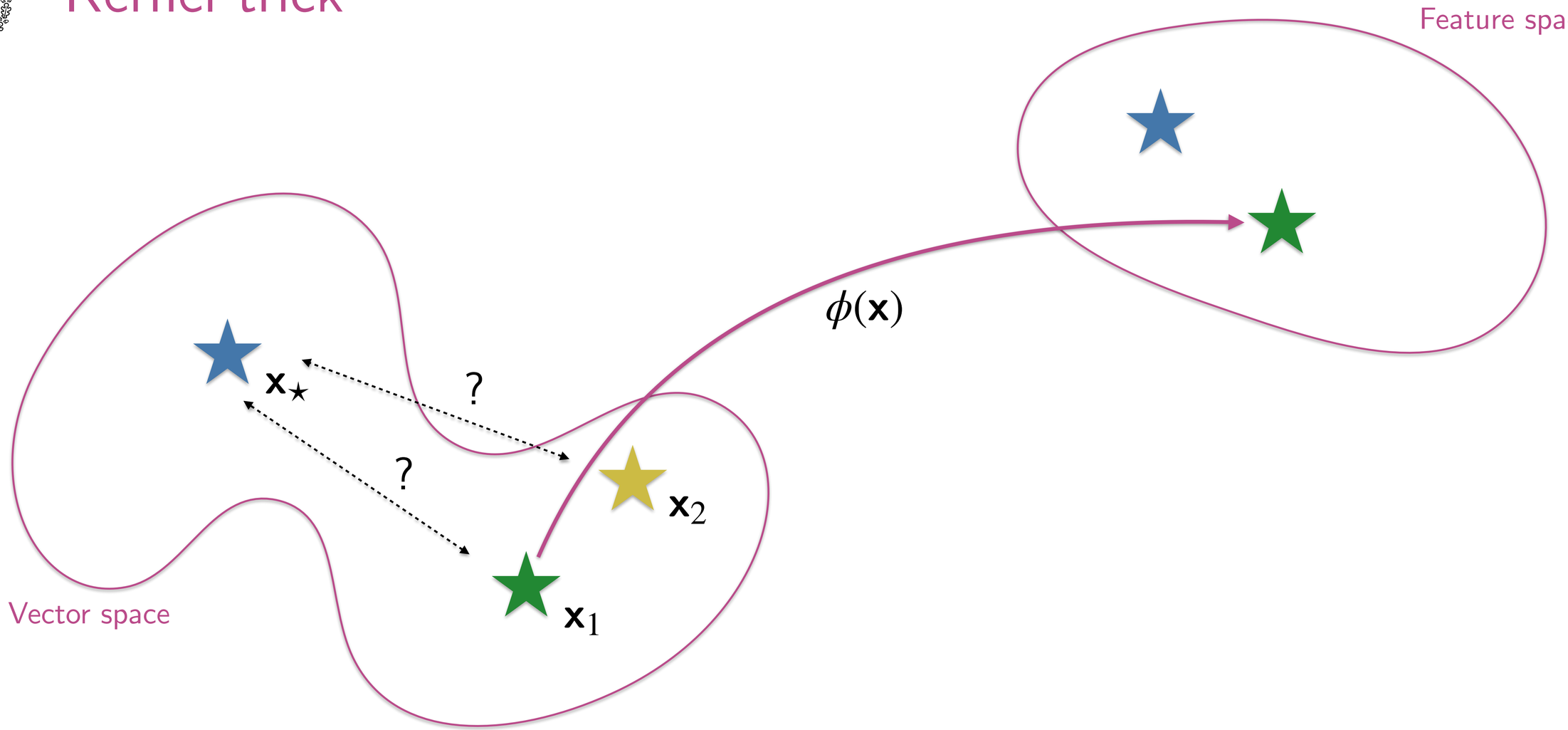
Kernel trick



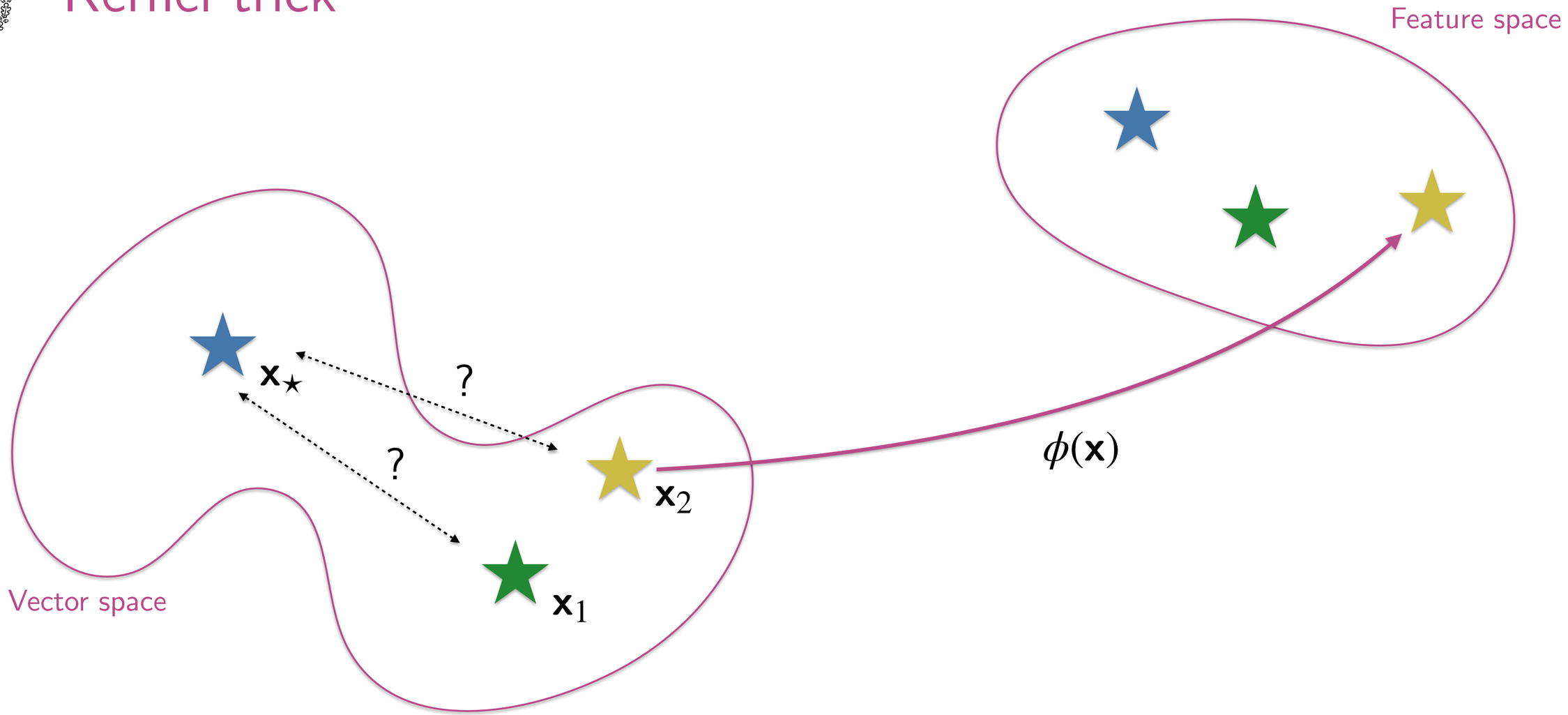
Kernel trick



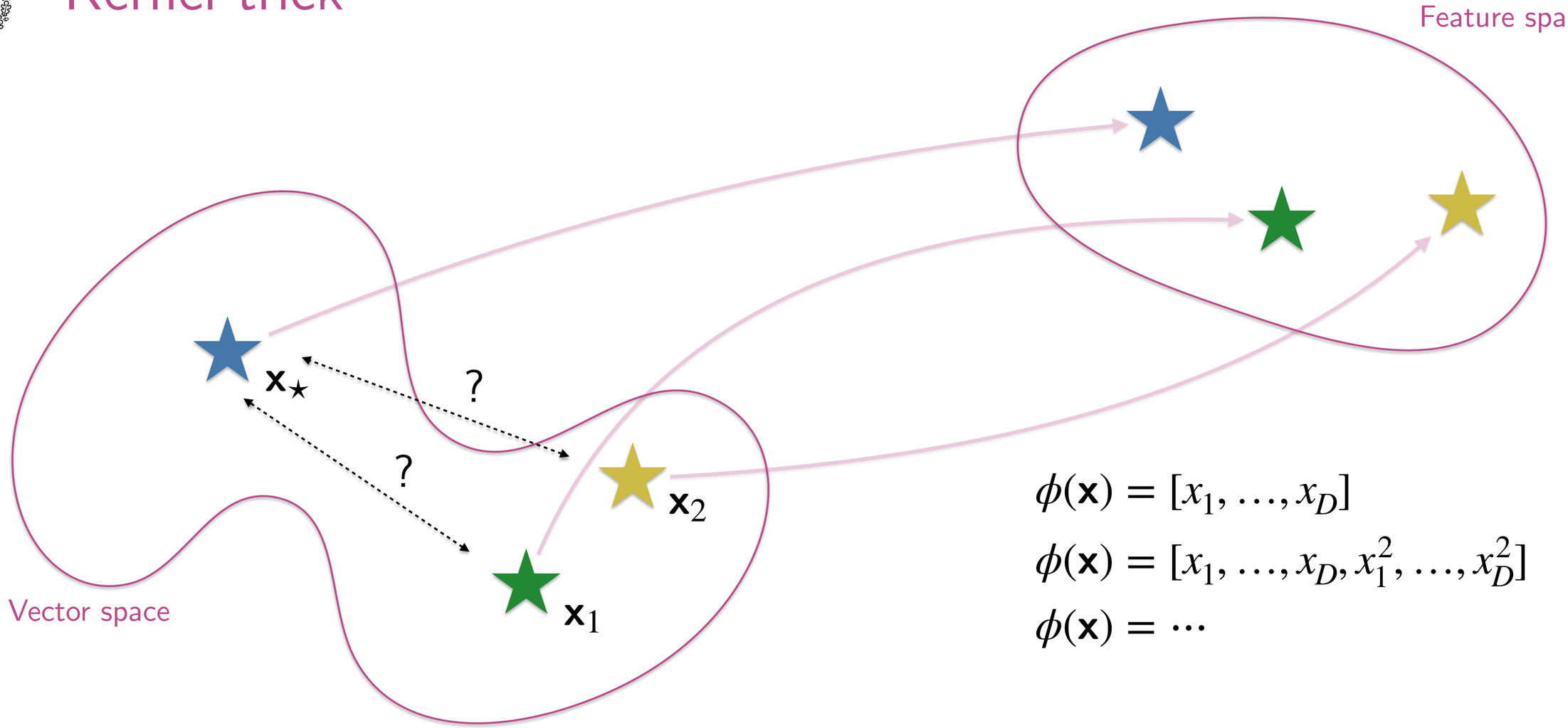
Kernel trick



Kernel trick



Kernel trick

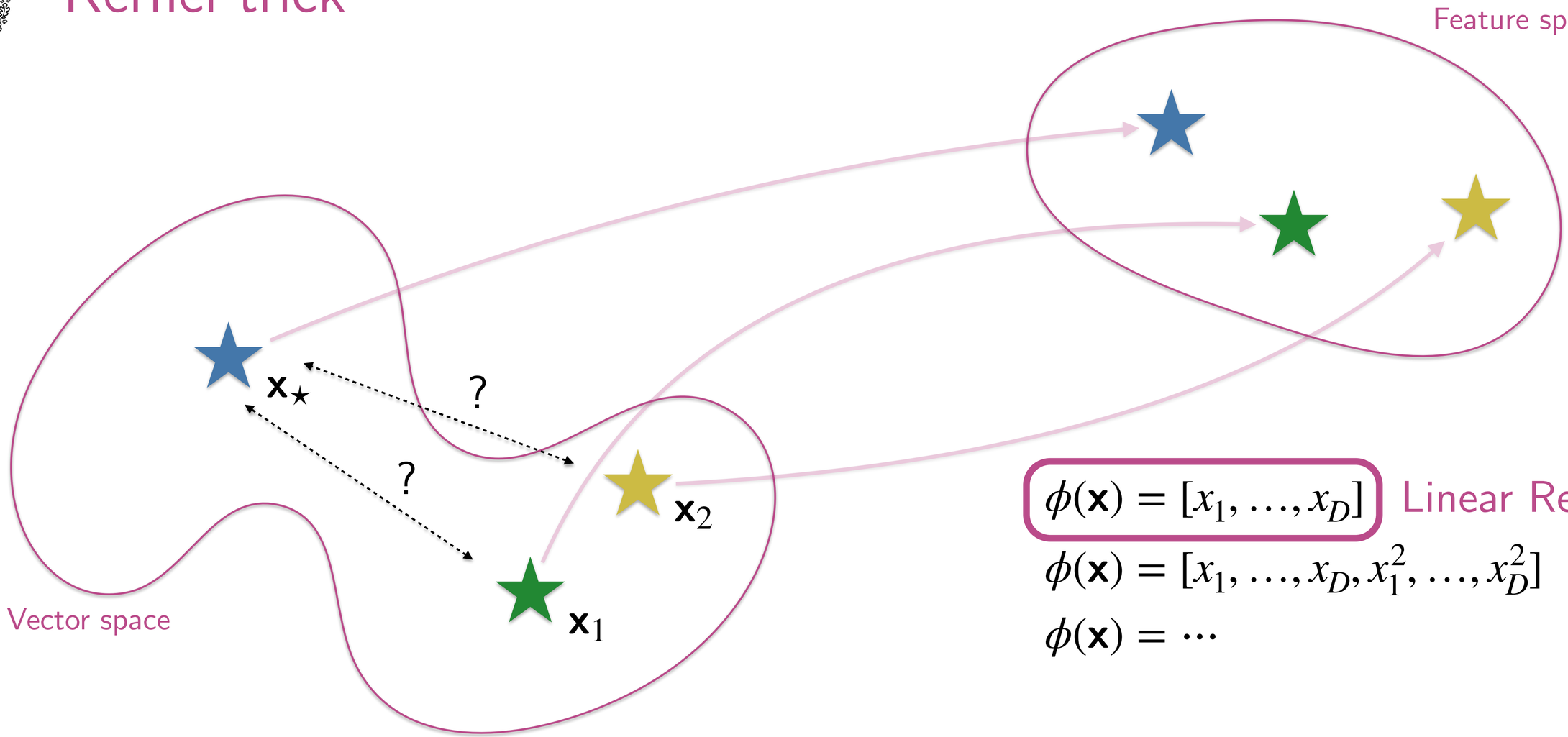


$$\phi(\mathbf{x}) = [x_1, \ldots, x_D]$$

$$\phi(\mathbf{x}) = [x_1, \ldots, x_D, x_1^2, \ldots, x_D^2]$$

$$\phi(\mathbf{x}) = \cdots$$

Kernel trick

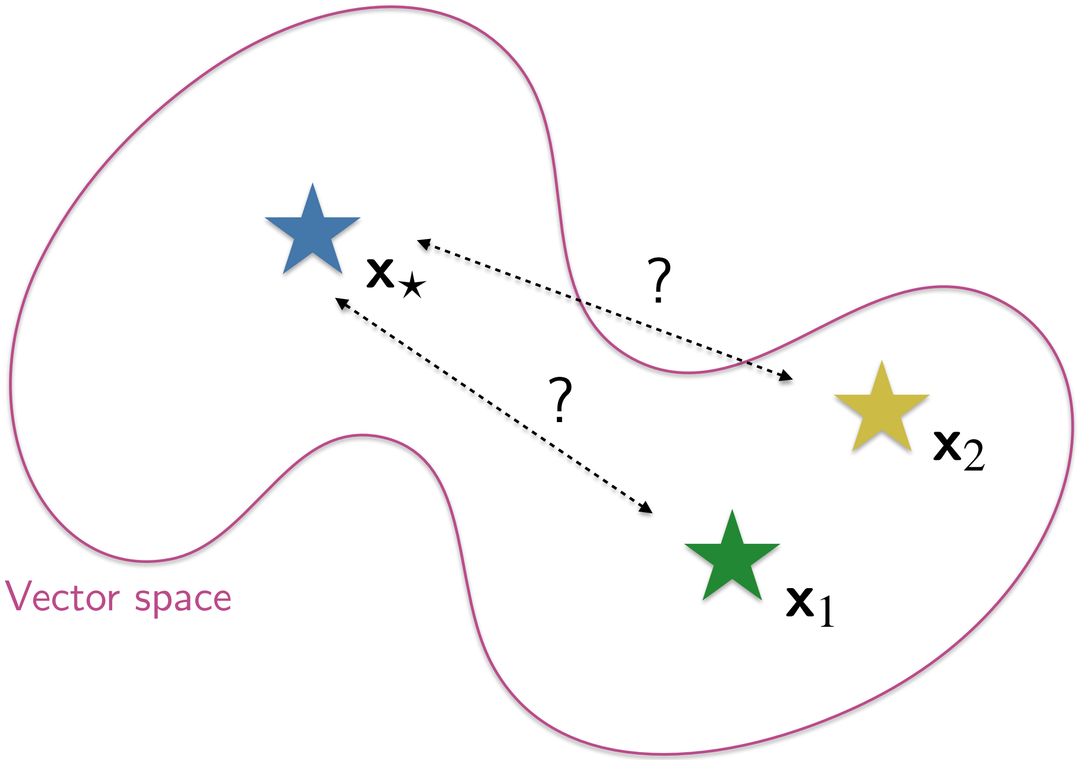


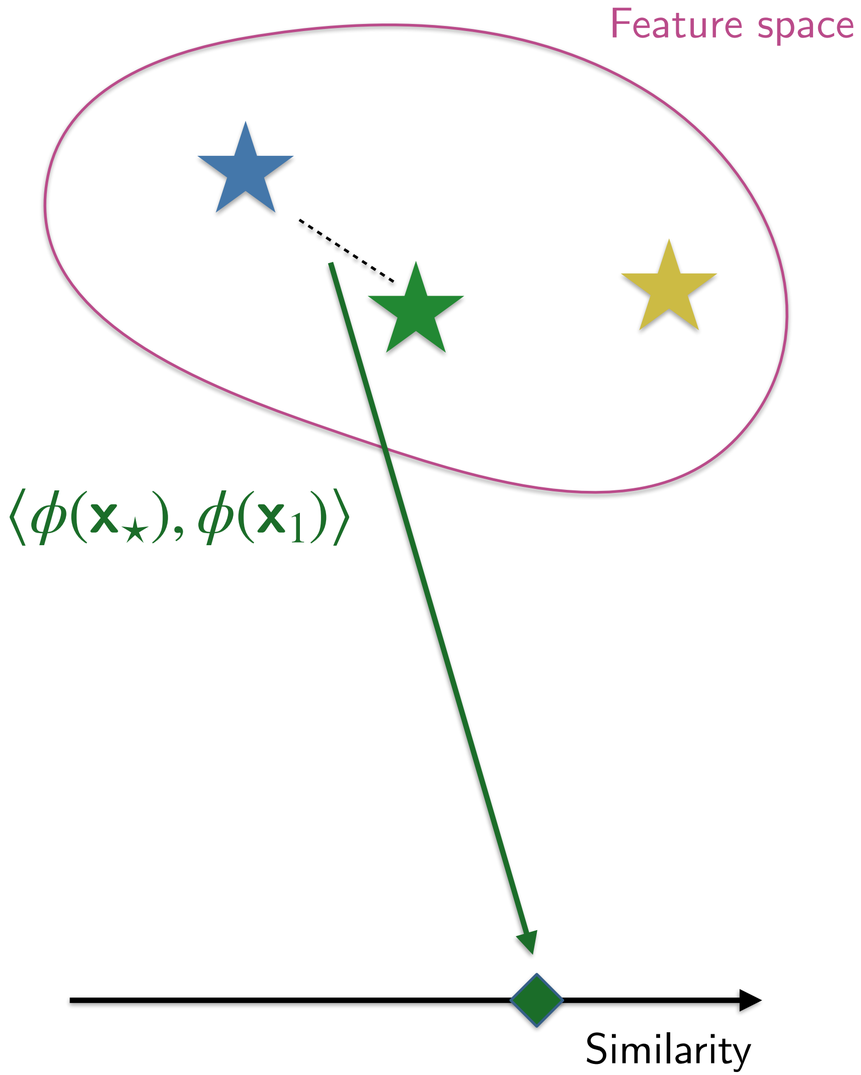
$\phi(\mathbf{x}) = [x_1, \ldots, x_D]$ Linear Regression

$\phi(\mathbf{x}) = [x_1, \ldots, x_D, x_1^2, \ldots, x_D^2]$

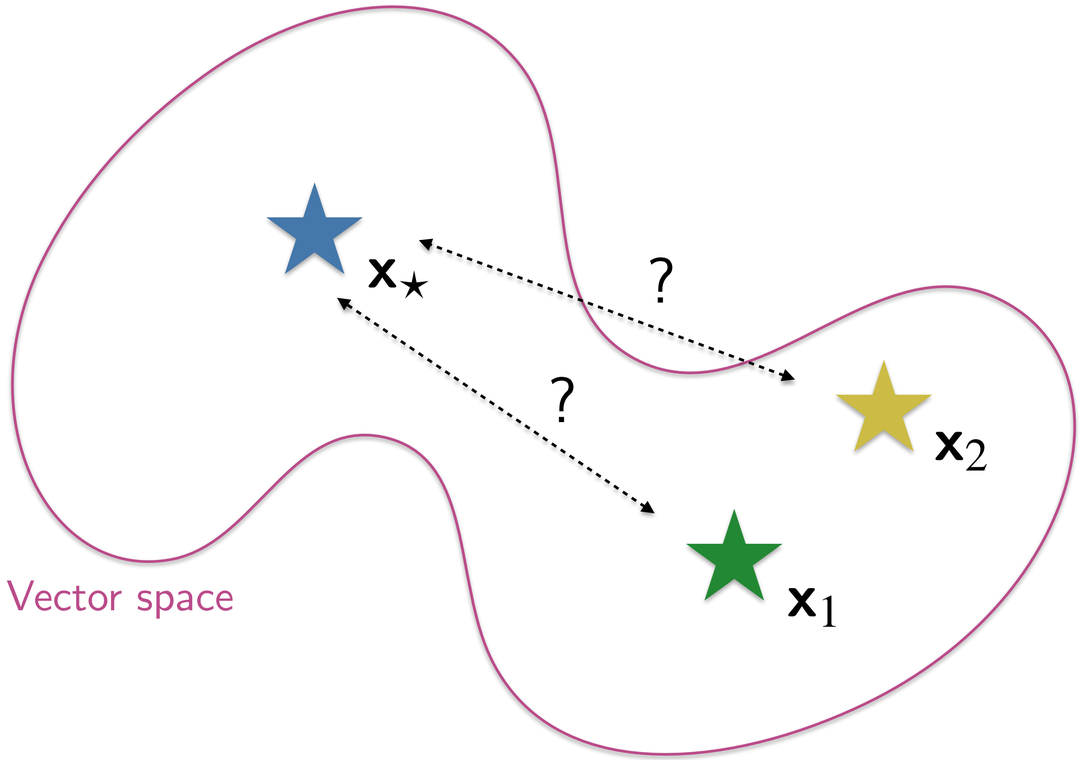
$\phi(\mathbf{x}) = \cdots$

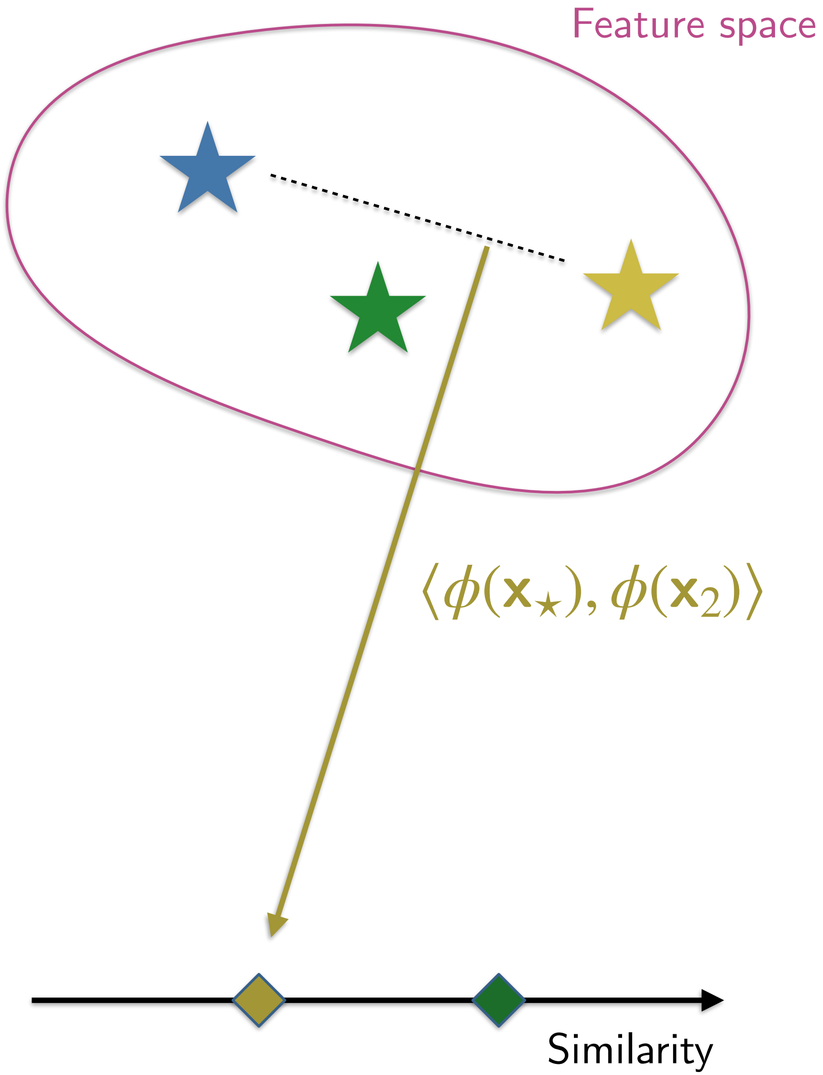
Kernel trick



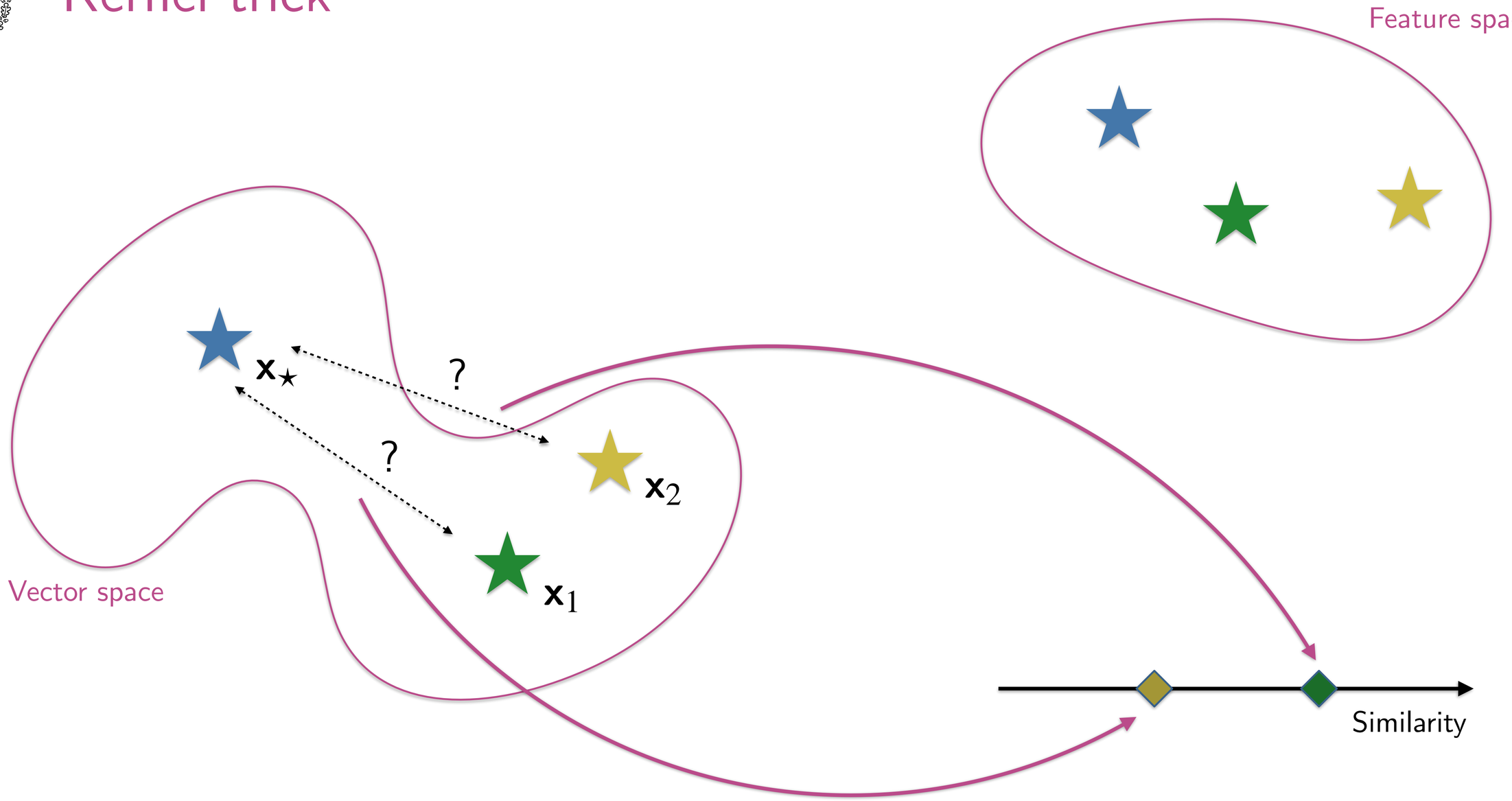


Kernel trick

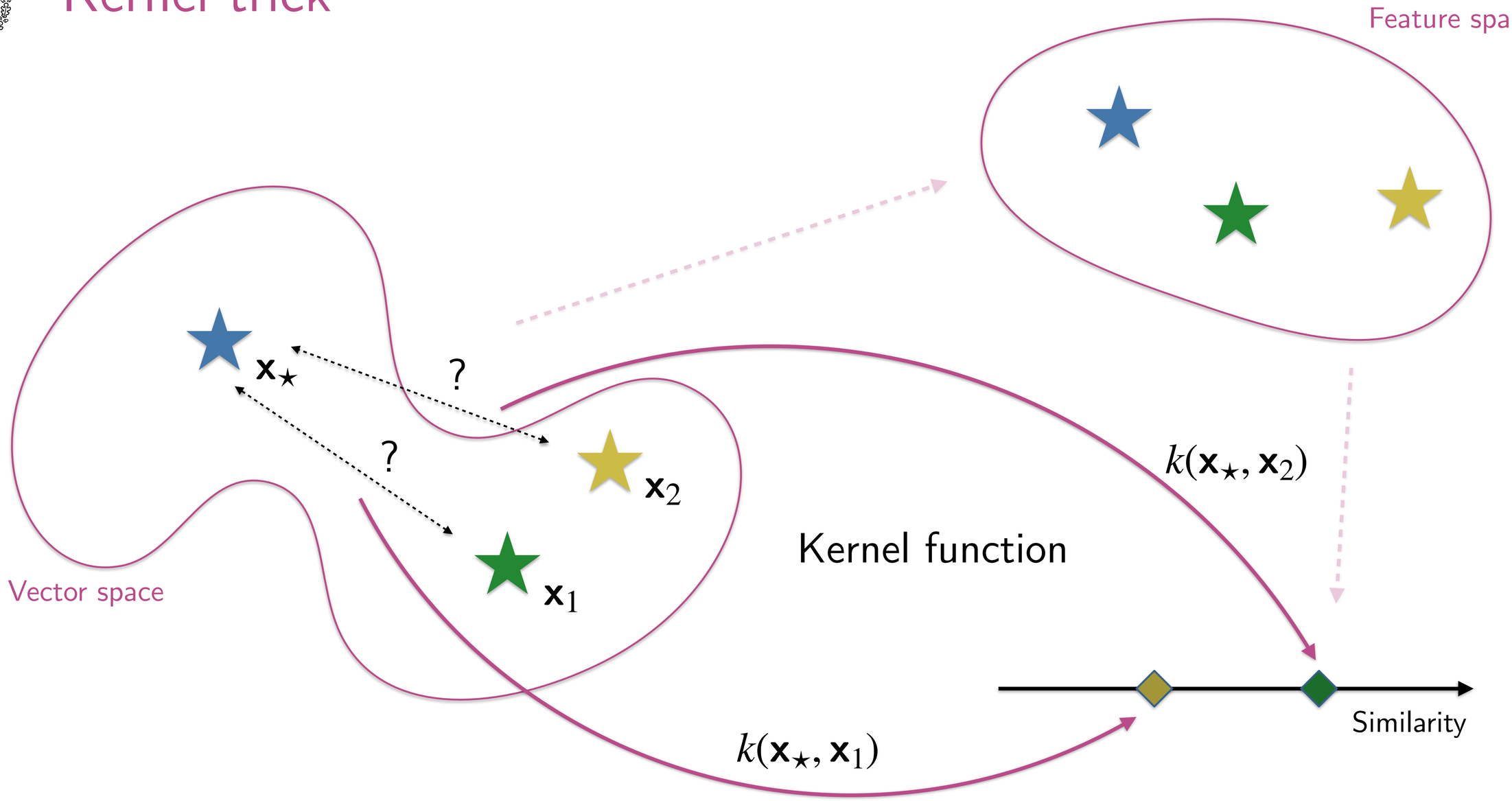




Kernel trick



Kernel trick



From Linear to Kernel Ridge Regression

- **Linear Regression:** f is *linear* in the parameters and inputs!

$$f(\mathbf{x}_\star) = \mathbf{x}_\star^T \mathbf{w} = \sum_{n=1}^{N} y_n \langle \mathbf{x}_\star, \mathbf{x}_n \rangle_{\mathbf{S}^{-1}} \qquad \mathbf{S} = \mathbf{X}^T \mathbf{X}$$

From Linear to Kernel Ridge Regression

- **Linear Regression:** f is *linear* in the parameters and inputs!

$$f(\mathbf{x}_\star) = \mathbf{x}_\star^T \mathbf{w} = \sum_{n=1}^{N} y_n \langle \mathbf{x}_\star, \mathbf{x}_n \rangle_{\mathbf{S}^{-1}} \qquad \mathbf{S} = \mathbf{X}^T \mathbf{X}$$

- **Kernel Regression:** f is *linear* in the parameters!

$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} y_n \langle \phi(\mathbf{x}_\star), \phi(\mathbf{x}_n) \rangle_{\mathbf{S}^{-1}} \qquad \mathbf{S} = \Phi^T \Phi$$

$$\Phi = \left[\phi(\mathbf{x}_1), \ldots, \phi(\mathbf{x}_N) \right]^T$$

From Linear to Kernel Ridge Regression

- **Linear Regression:** f is *linear* in the parameters and inputs!

$$f(\mathbf{x}_\star) = \mathbf{x}_\star^T \mathbf{w} = \sum_{n=1}^{N} y_n \langle \mathbf{x}_\star, \mathbf{x}_n \rangle_{\mathbf{S}^{-1}} \qquad \mathbf{S} = \mathbf{X}^T \mathbf{X}$$

- **Kernel Regression:** f is *linear* in the parameters!

$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} y_n \langle \phi(\mathbf{x}_\star), \phi(\mathbf{x}_n) \rangle_{\mathbf{S}^{-1}} \qquad \mathbf{S} = \Phi^T \Phi$$

$$\Phi = \left[\phi(\mathbf{x}_1), \ldots, \phi(\mathbf{x}_N) \right]^T$$

Problem: $\phi(\,\cdot\,)$ needs to be known explicitly but is infinite-dimensional for some kernels

From Linear to Kernel Ridge Regression

- **Kernel Regression:** f is *linear* in the parameters!

$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} y_n \langle \phi(\mathbf{x}_\star), \phi(\mathbf{x}_n) \rangle_{\mathbf{S}^{-1}}$$

$$\mathbf{S} = \Phi^T \Phi$$

$$\Phi = \left[\phi(\mathbf{x}_1), \ldots, \phi(\mathbf{x}_N) \right]^T$$

From Linear to Kernel Ridge Regression

- **Kernel Regression:** f is *linear* in the parameters!

$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} y_n \langle \phi(\mathbf{x}_\star), \phi(\mathbf{x}_n) \rangle_{\mathbf{S}^{-1}}$$

$$\mathbf{S} = \Phi^T \Phi$$

$$\Phi = \left[\phi(\mathbf{x}_1), \ldots, \phi(\mathbf{x}_N)\right]^T$$

- **Solution:** Use the kernel trick!

$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} y_n \langle \phi(\mathbf{x}_\star), \phi(\mathbf{x}_n) \rangle_{\mathbf{S}^{-1}} = \sum_{n=1}^{N} y_n \, \phi(\mathbf{x}_\star)^T \left(\mathbf{\Phi}^T \mathbf{\Phi}\right)^{-1} \phi(\mathbf{x}_n)$$

$$= \sum_{n=1}^{N} k(\mathbf{x}_\star, \mathbf{x}_n) \left[\mathbf{K}^{-1}\mathbf{y}\right]_n \qquad [\mathbf{K}]_{nm} = k(\mathbf{x}_n, \mathbf{x}_m)$$

Implementing Kernel Ridge Regression

Kernel Regression

```python
import numpy as np

...

# normalize (keep the statistics for prediction)
X_mean, X_std, y_mean = X.mean(axis=0), X.std(axis=0), y.mean()
y = y - y_mean
X = (X - X_mean) / X_std

def rbf_kernel(a, b, lengthscale=1.0):
    return np.exp(-0.5 * np.sum((a - b)**2) / lengthscale**2)

def gram_matrix(A, B):
    return np.array([[rbf_kernel(a, b) for b in B] for a in A])

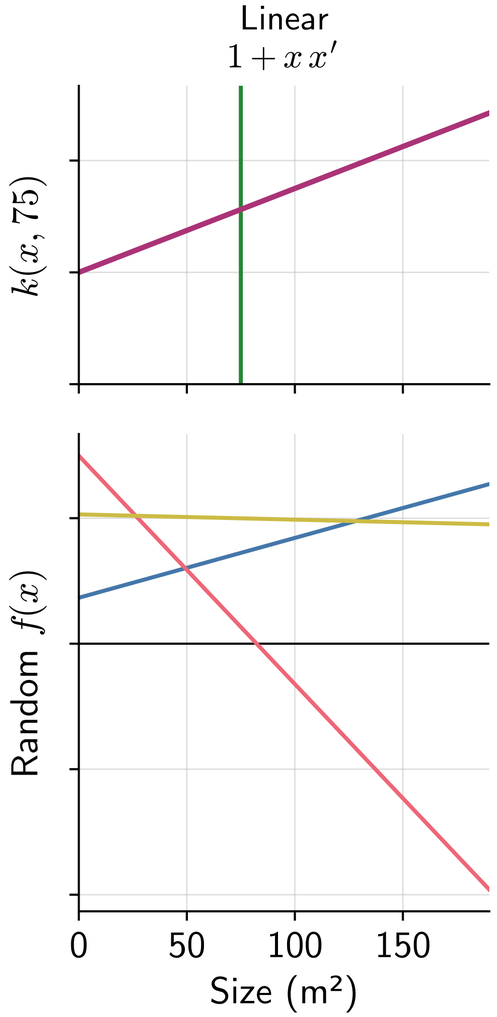
K = gram_matrix(X, X)  # N x N Gram matrix
beta = np.linalg.solve(K, y)

X_star = (np.array([[3.0, 40.0]]) - X_mean) / X_std  # new input, same scaling
k_star = gram_matrix(X_star, X)                      # M x N

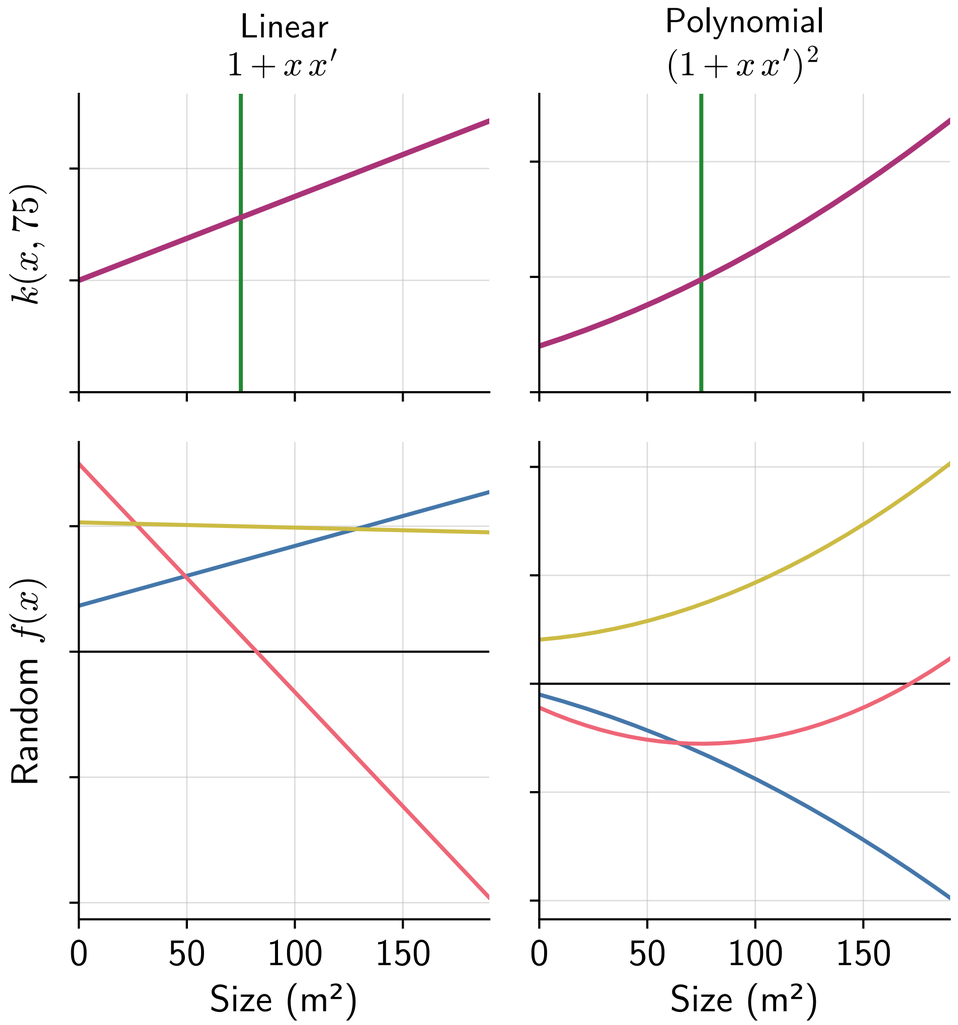
f_star = k_star @ beta + y_mean
```

https://github.com/traml-lab/id2214-lectures/blob/main/lecture5-kernel-regression.py

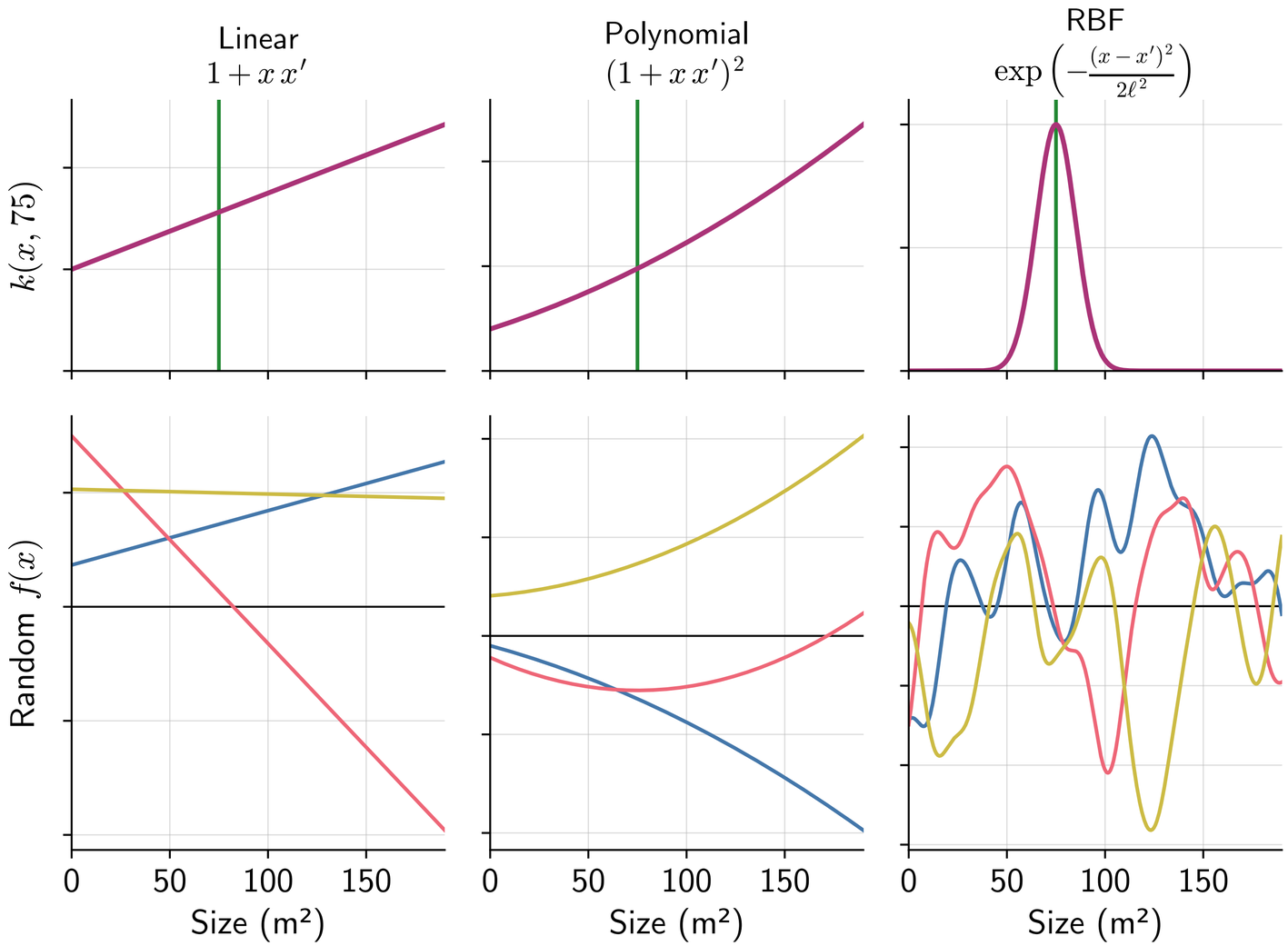
Kernel functions



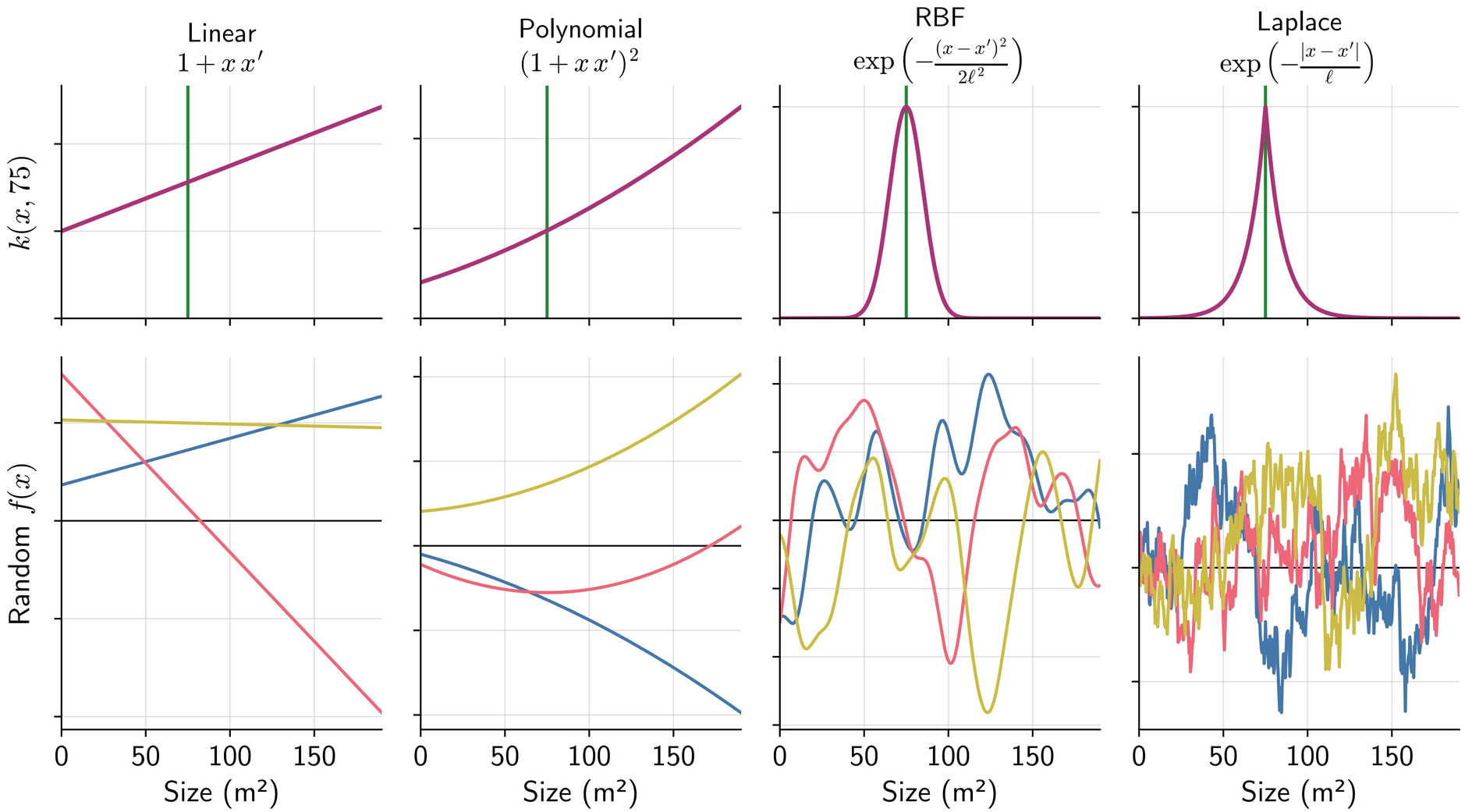
Kernel functions



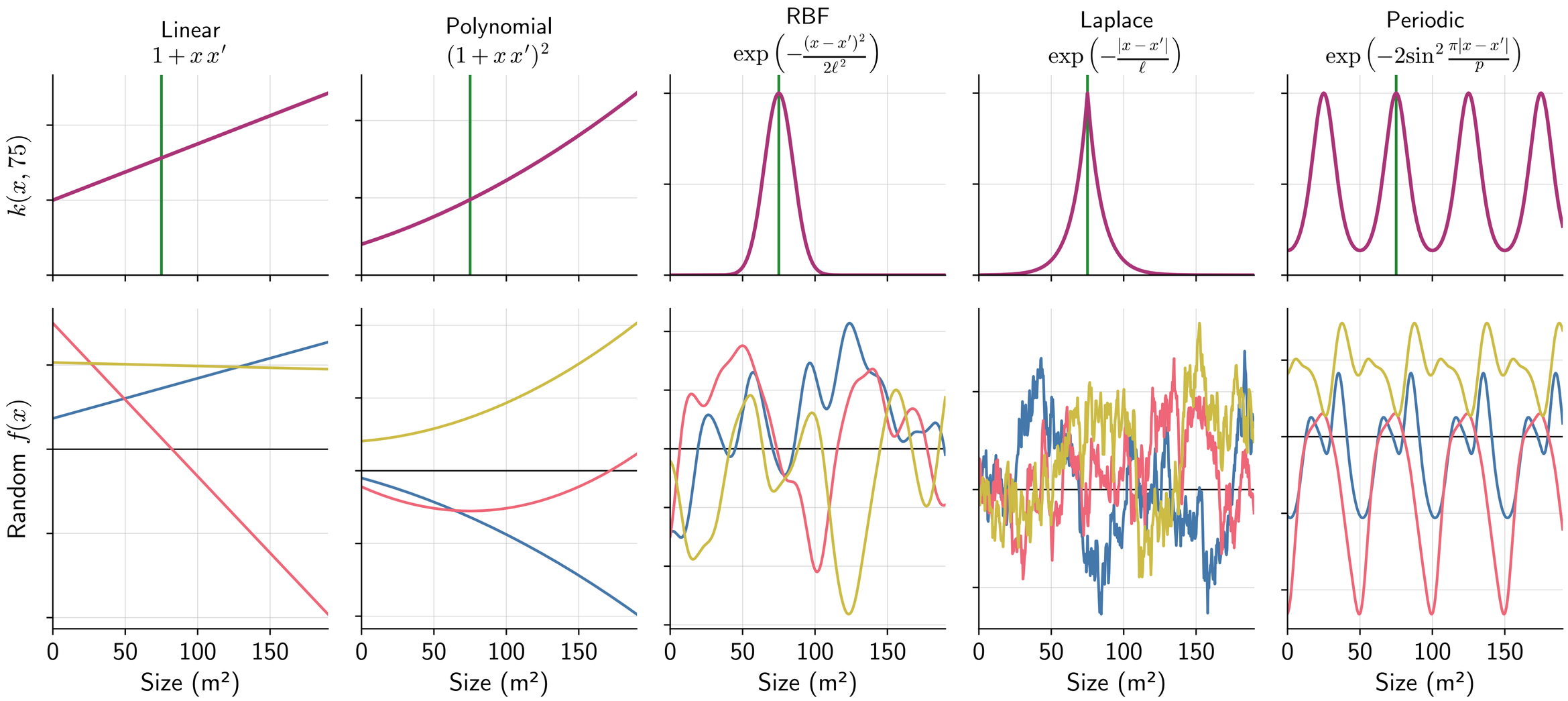
Kernel functions



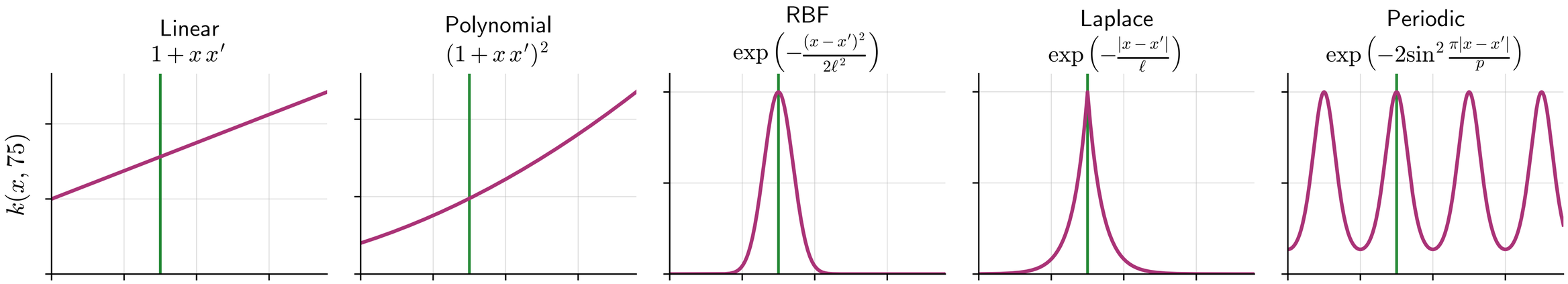
Kernel functions



Kernel functions



Kernel functions

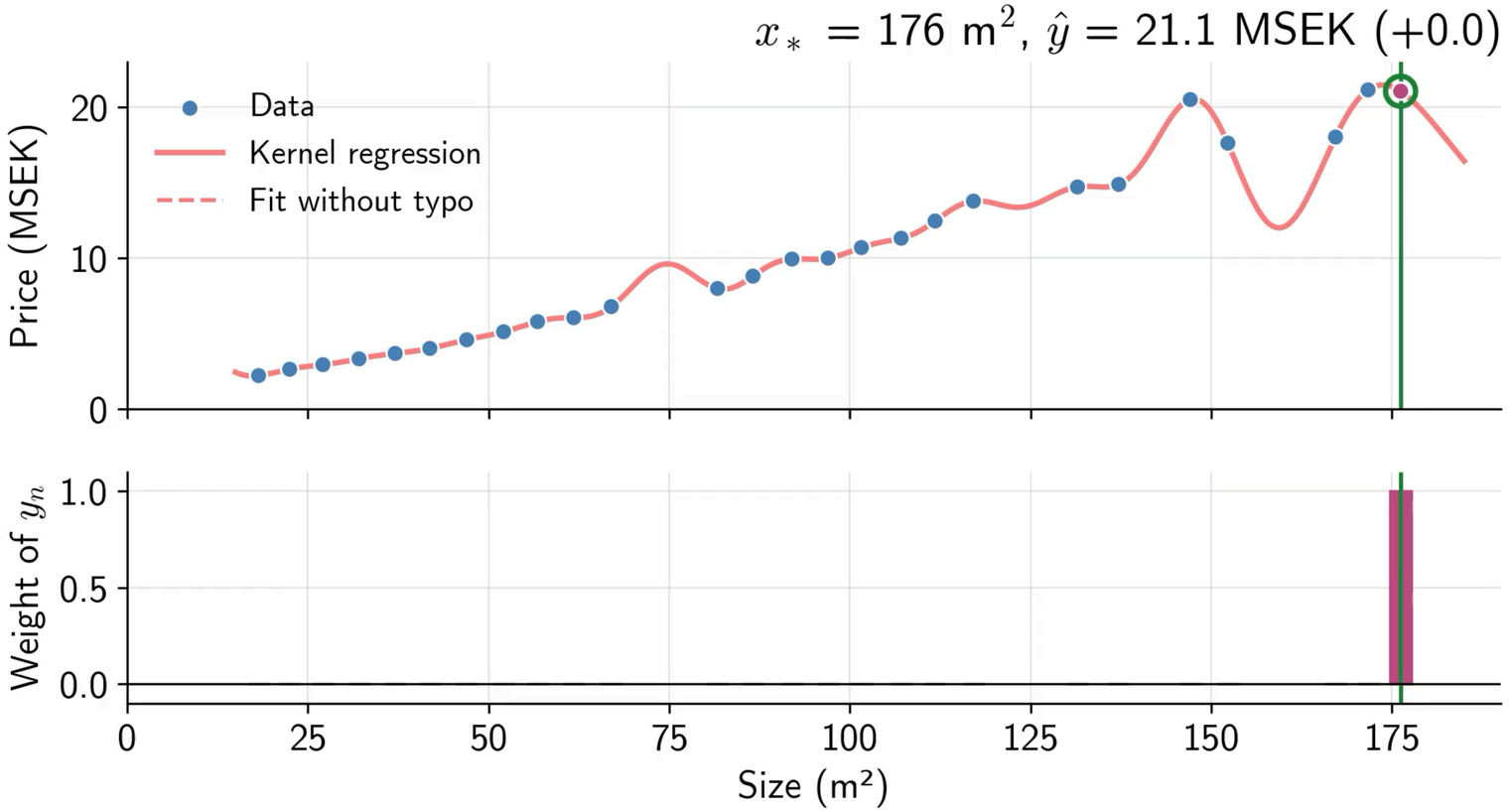


- Which **kernel function** to use is a **design choice**!

$$\sum_{n=1}^{N} k(\mathbf{x}_\star, \mathbf{x}_n)\left[\mathbf{K}^{-1}\mathbf{y}\right]_n$$

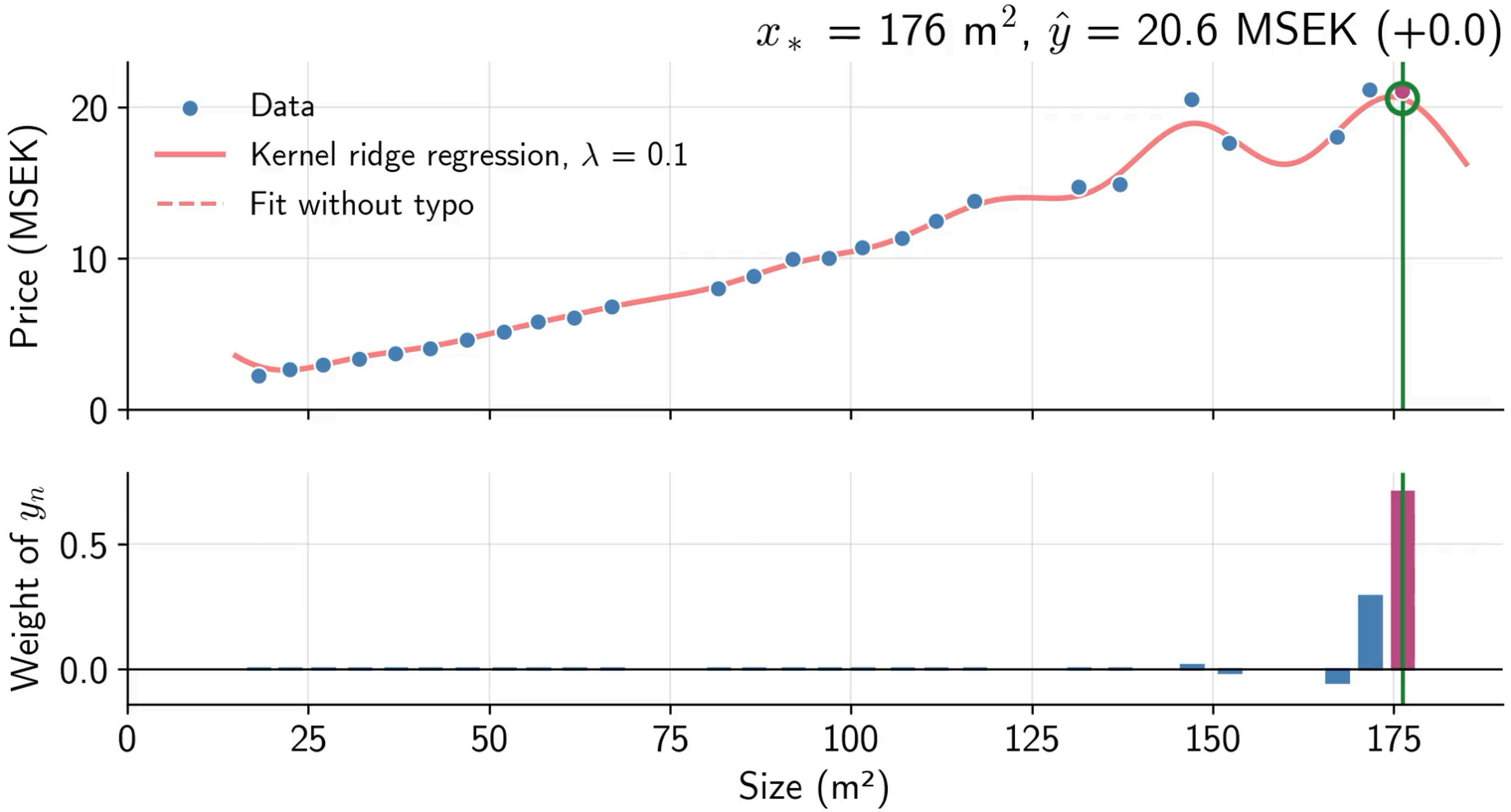
Motivation for Kernel Ridge Regression

- **Problem:** If the kernel is "too flexible", we can overfit to outliers



Motivation for Kernel Ridge Regression

- **Solution:** Regularise to protect against overfitting (kernel ridge regression)



From Linear to Kernel Ridge Regression

- **Linear Regression:** f is *linear* in the parameters and inputs!

$$f(\mathbf{x}_\star) = \mathbf{x}_\star^T \mathbf{w} = \sum_{n=1}^{N} y_n \langle \mathbf{x}_\star, \mathbf{x}_n \rangle_{\mathbf{S}^{-1}} \qquad \mathbf{S} = \mathbf{X}^T \mathbf{X}$$

- **Kernel Regression:** f is *linear* in the parameters!

$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} k(\mathbf{x}_\star, \mathbf{x}_n) \left[\mathbf{K}^{-1}\mathbf{y}\right]_n \qquad [\mathbf{K}]_{nm} = k(\mathbf{x}_n, \mathbf{x}_m)$$

From Linear to Kernel Ridge Regression

- **Linear Regression:** f is *linear* in the parameters and inputs!

$$f(\mathbf{x}_\star) = \mathbf{x}_\star^T \mathbf{w} = \sum_{n=1}^{N} y_n \langle \mathbf{x}_\star, \mathbf{x}_n \rangle_{\mathbf{S}^{-1}} \qquad \mathbf{S} = \mathbf{X}^T \mathbf{X}$$

- **Kernel Regression:** f is *linear* in the parameters!

$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} k(\mathbf{x}_\star, \mathbf{x}_n) \left[\mathbf{K}^{-1}\mathbf{y}\right]_n \qquad [\mathbf{K}]_{nm} = k(\mathbf{x}_n, \mathbf{x}_m)$$

- **Kernel Ridge Regression**: f is *linear* in the parameters and regularised!

$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} k(\mathbf{x}_\star, \mathbf{x}_n) \left[(\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y}\right]_n$$

The End

Next Lecture: A probabilistic view on kernel regression

